DOINGG@UNION.EDU

COURSE: CSC106 CAN COMPUTERS THINK?

SELF

Contents

Home 5

Announcements 5

Course Description 6

Schedule 9

Schedule 9

Resources 11

CS-Ticket 11

Runestone 11

Gradescope 11

Late Token Form 11

Python 12

Python Documentation 12

Reeborg 12

PythonTutor 12

Google Colab Pro 12

Python Style Guide 12

Syllabus 13

Grading 27

Assignments 29

PF 01 31

CSC106: Portfolio 1 33

PF 02 57

CSC106: Portfolio 2 59

Project 1 85

CSC106: Project 1 87

PF 03 95

CSC106: Portfolio 3 97

PF 04 119

CSC106: Portfolio 4 121

Project 2 145

CSC106: Project 2 147

PF 05 155

CSC106: Portfolio 5 157

Weekly Notes 181

Week 1 183

Week 2 185

Examples 186

Week 3 187

Examples 188

Week 4 189

Week 5 191

Week 6 193

About 195

Me: 197

Home



The course website for CSC106, part of the Union College CS curriculum.
Here you can find our weekly schedule, assignmnets and resources.

Announcements

- Midtern exam tomorrow Thurs 10/15
  - hand-written, in **pen** unless accomodation needed
  - can use scrap paper, if needed
  - 1.5 hours for 1x time
  - if needed, make sure you have a testing center spot

- see week6 notes for review Qs

Office Hours

- Student/Office Hours:
  - Steinmetz 217
  - Tuesday 2:00 pm - 3:30 pm
  - Thursday 2:00 pm - 3:30 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/8bus6pfDvyphR9ar5
  - by appointment for another time or over zoom

Course Description

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120), to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior' ' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing

in whatever area you are interested in. And if you're here just because you're curious, that's great!

Schedule

Weekly Schedule

Schedule

check often as things may change

W	TOPIC	Due by end of week	Runestone
1	Programming, Syntax and Semantics	Portfolio (PF) 1	1 & 2
2	Variables and Functions	PF 2	3 & 4
3	Conditional Statements	PF 3, Project Demo 1	5
4	Conditional Loops	PF 4, Practical Exam 1	6
5	Counting Loops	PF 5, Project Demo 2	7
6	Strings, Lists	PF 6, Written Exam 1	8 & 9
7	Nested Lists, Dictionaries	PF 7, Project Demo 3	10
8	File I/O	PF 8, Practical Exam 2	11
9	Recursion	PF 9, Project Demo 4	
10	Searching, Sorting	PF 10	

W	TOPIC	Due by end of week	Runestone
11	Finals Week Monday 11/23 2:30 - 4:30	Written Exam 2	

Resources

CS-Ticket

- technical help with lab machines
- https://csticket.union.edu/

Runestone

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_foppff_fall26 as the course name

Gradescope

- https://www.gradescope.com/courses/1370118
- Entry Code: 3ZGEVN

Late Token Form

- https://forms.gle/bKTWFgftoT9sYFxJ7
- fill out this form to get a 24 hour extension on any Programming Project
- each student gets 3 tokens per term

Python

- Python 3.13.X: the software package needed to write and run Python programs
- https://www.python.org/downloads/

Python Documentation

- Official Documentaion: https://docs.python.org/3/index.html
- Can *always* be used as a reference
- Can be searched for module-specific documentation e.g. turtle

Reeborg

- Reeborg's World: a useful website for learning some programming basics
- Reeborg's World: https://reeborg.ca/reeborg.html
- Reeborg Project Documentation

PythonTutor

- Python Tutor: a useful website for tracing python code
- https://pythontutor.com/visualize.html#mode=edit

Google Colab Pro

- Pro account sign-up: https://colab.research.google.com/signup
- Use your Union email
- A pro account is not needed for this course, just an option Union pays for
- Notes template: https://colab.research.google.com/drive/15rPmKyQKCCbwsdDCpC39o8hczWZ-B5zV?usp=sharing

Python Style Guide

- PEP 8
- should be used in all code cells of Colab notebooks
- Link: https://peps.python.org/pep-0008/

Syllabus

Syllabus

- check for updates! (All updated text in this posted version of the syllabus will supersede text from older versions.)

CSC106 Syllabus

1 CSC 106: Can Computers Think?

- Union College, Fall 2026
- Georgia Doing, PhD: email: doingg@union.edu, office: Steinmetz 217

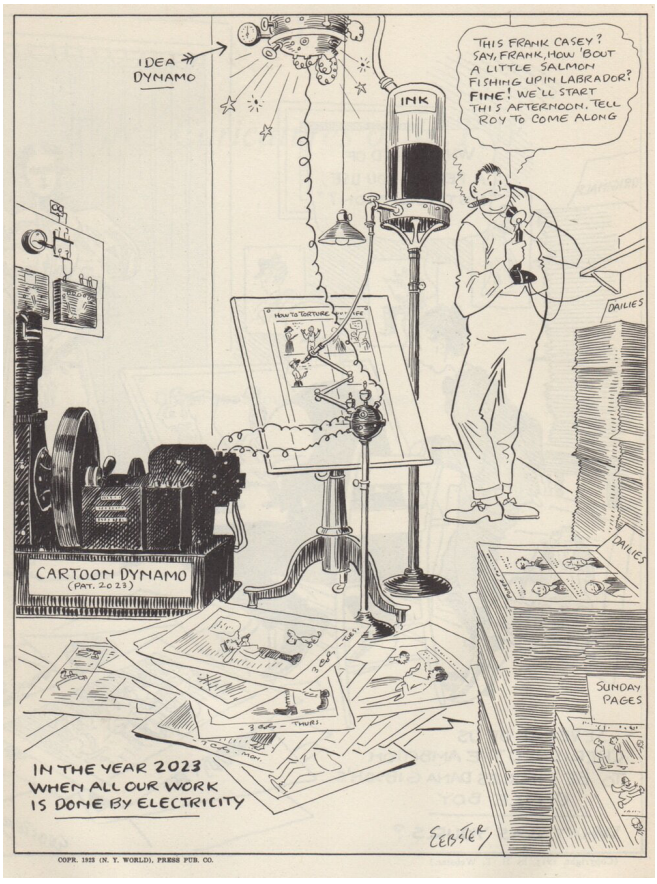


Figure 1: Cartoon by H.T. Webster published in 1922.

2 COURSE BASICS

- Lecture: Monday, Wednesday, Friday 1:50 pm - 2:55 pm
- Lab: Thursday 10:55 am - 12:40 pm
- Location: Olin 107
- Student/Office Hours:
  - Steinmetz 217
  - Monday 3:00 pm - 4:30 pm
  - Thursday 2:00 pm - 3:30 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/3kgVY1utGu5Mtzz7A
  - by appointment for another time or over zoom

3 OVERVIEW

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120), to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior' ' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

4 LEARNING OBJECTIVES

By the end of the course, you should be able to *answer* the following questions:

- What are the most important programming fundamentals and why are they the building blocks of programs?
- What are the debugging fundamentals and how do you use them?
- What are some of the mechanisms used to build "intelligent" machines?

By the end of the course, you should be able to *do* the following:

- Use variables, functions, conditionals, loops and recursion
- Read Python programs
- Write short Python programs that solve a given problem
- Fix short Python programs using debugging fundamentals

Computing/programming topics we will cover include functions, variables, assignment, conditionals, loops, recursion, lists/arrays, file reading/writing, binary numbers and ASCII, strings and string methods and dictionaries.

A 1-page schedule-at-a-glance is included at the end of the syllabus, but please see Nexus for a link to a more detailed schedule.

5 CLASS POLICIES AND GUIDELINES

The Computer Science Department as a whole welcomes all people, regardless of age, background, beliefs, ethnicity, gender identity, gender expression, national origin, religious affiliation, sexual orientation and any other differences, be they visible or non-visible. It's also important to recognize that institutional racism has prevented members of marginalized groups - especially black, indigenous and other people of color (BIPOC) - from fully participating in the field of Computer Science.

You. Yes you, the one reading this syllabus. You belong here.

As an instructor, I will do my utmost to uphold these principles and to treat everyone with respect. As students in this class, I expect you to do the same since one person is not a community unto themselves. We're all in this together, so let's treat each other well. Lastly, I welcome feedback on issues we might discuss or ways that our class can be a more just one for all people.

6 COURSE MATERIALS

Online Textbook: Foundations of Python Programming Functions First - Interactive (free).

We are going to use an online, interactive, free and open source textbook. This textbook has a lot of embedded interactive exercises. Reading this book always also means doing the activities. To get access to the book, please register on Runestone using the following informationm which can also be found on Nexus:

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_foppff_fall26 as the course name

We'll use the following websites and software throughout the course:

- The course website: a github-hosted site with assignments, materials and announcements
  - https://georgiadoing.github.io/CSC106-F26/
  - you may want to bookmark this to check it before and after each class
- Gradescope: used to submit programming assignments
  - https://www.gradescope.com/courses/1370118
  - Entry Code: 3ZGEVN
- Python 3.13.X: the software package needed to write and run Python programs
- Reeborg's World: a useful website for learning some programming basics
- https://reeborg.ca/index_en.html
- Python Tutor: a useful website for tracing python code
  - https://pythontutor.com/visualize.html#mode=edit

7 COURSE RESOURCES

The course website has a list and links to the readings and software resources that we are going to use. In class, we are going to use the Linux machines in Olin 107. Outside of class you have a choice or hardware. You can install the software on your own computer (Python is freely available for Windows, Mac, and Linux, Reeborg's World and Python Tutor are browser based programs) or you can work in one of the Computer Science labs. We have three spaces that you can use 24/7 using your ID card, except when classes are being held in them:

- Olin 107 - where we have class
- PASTA Lab (ISEC 051+)
- Computer Science Resource room (Steinmetz Hall, 209A)

8 LIBRARY RESOURCES

The library is available to help you with your research needs! Librarians can help you develop research questions, search for and select the best sources for your projects, identify research strategies, evaluate sources, and assist you with creating citations. There are multiple ways for you to contact a librarian. For more information, please see our Ask A Librarian page: https://www.union.edu/schaffer-library/ask-a-librarian.

9 ACCOMODATIONS

It is the policy of Union College to make reasonable accommodations for qualified individuals with disabilities. If you are a person with a disability and wish to request accommodations to complete your course requirements, please make an appointment with me or stop by during my office hours as soon as possible, all discussions will remain confidential. You must provide reasonable notice and be in touch with Accomodative Services on the 2nd floor of Schaffer Library, if you have not already, if you wish to take advantage of extra time on exams.

10 COMPLEX QUESTIONS: GLOBAL CHALLENGES & SOCIAL JUSTICE

Justice, Equity, Identity, Difference (JEID). Algorithms and computational systems increasingly shape the world we live in. Advances in AI and machine learning are in the news on a daily basis. But there is also a growing awareness of the limitations and dangers of these systems. In particular, they often promote biases and inequalities. For example, facial recognition software has been shown to work much less accurately for people of color than for white people, an AI hiring algorithm was found to reject female applicants for technical positions at a higher rate than male applicants, and search results can be full of racial stereotypes. In this course, you will learn about real-world cases of AI systems that discriminate against groups of people. You will see that related ethical questions are raised even in the context of the relatively simple programs you create in an introductory computer science class. Additionally, you will work on developing a habit of always evaluating how your own choices as a programmer might impact other people's experiences.

Engineering, Technology and Society (ETS). In this course, you learn to break down a larger problem into a series of smaller computational steps (creating an algorithm) and to implement algorithms in Python. Through these activities you will gain an understanding of how computers work and an idea of the software development process. In particular, you will learn about iterative development and the importance of thorough testing. You will practice communicating about algorithms and their implementation in groups.

You will also explore, through assignments and discussion, how computational systems can have unintended and negative consequences for groups of people and what the responsibility of the programmer is to mitigate these dangers.

Data & Quantitative Reasoning (DQR). Much of current AI systems are data driven and learn based on collections of data. In this course, you will learn about and implement some approaches that AI systems use to draw inferences based on data. Many of the ethical issues around current AI systems are linked to the data these systems use. Throughout the course, you will, therefore, also reflect on: What information is collected? How is the information represented? Who is represented in the data? How is it used?

11 GRADING

The following components will contribute to your final grade for this class with roughly the indicated weight.

- 2 practical exams: 20% (10% each)
- Written midterm: 20%
- Written final exam: 20%
- Programming project demos: 20%
- Portfolios & share-outs: 15%
- Reading & practice: 5%

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Letter Grade Scale:

- A: 93-100; A-: 90-92
- B+: 87-89; B: 83-86; B-: 80-82
- C+: 77-79; C: 73-76; C-: 70-72
- D: 60-69; F: 0-59

12 ATTENDANCE

Class participation is a critical component of the course and attendance is mandatory. I realize that sometimes other things come up (interviews, athletic events, illness, emergencies etc.). In those cases, just let me know (in advance, if at all possible) that you are going to be absent. There may not always be the possibility of make-up credit for missed in-class material (share-outs), but it is more likely for something to be arranged if you discuss your absence with me prior to class. No matter how much class you have missed, please do not come to class if you have an illness that is likley contageous, please email me to work out a reasonable solution.

If you do miss class, it is your responsibility to make up any material that you missed. Get notes from a classmate, make sure to complete all assignments due or assigned during the class that you missed, and come see me if you have any questions on the material. I will expect you to hand in all assignments by the due date, regardless of whether you were in class. You will not be able to make up missed exams or in-class share-outs without pre-approved exceptions.

13 LATE POLICIES

Portfolio assignments will usually be discussed in class on the day that they are designated, portfolios will be collected the day they are due and will not be accepted late. Programming projects are due by 11:59:59pm on the due date. You have three "extension tokens" for this class. Each token will extend a single project deadline by 24 hours. You can use each token on a different project, or use two or even all three on a single project. Once all three tokens have been used, no late submissions will be accepted (barring exceptional circumstances).

14 PARTICIPATION AND ENGAGEMENT

Following these guidelines will constitute positive participation and engagement and earn you **up to 1.5% of your final grade per week**. To demonstrate comprehension of in-class and/or homework assignments and earn relevent engagement points through share-outs (0.5% per week), students will submit written answers for collection as portfolios (1% per week) and adhere to the follwoing guidelines. **Falling short of any of the below standards may result in point deductions from any engagement activities relevent to class time(s) during which the student was unengaged**.

- Respect. This course is a space for rigorous and respectful debate. When confronting ideas and people different from you, lead with curiosity rather than judgement. We will

critique ideas, not people. To protect our shared learning environment, you may not record, photograph, or share any part of our class sessions outside of our class.

- Engage. Show up to class on time and prepared. Show up ready to learn by completing the readings and exercises and checking the course website often. Engagement in class constitutes being present, respectful and lending the class your full attention. **It is the student's responsibility to demonstrate this engagement by completing and handing-in thoughtful written responses as directed by in-class prompts and/or homework assignments.** Accessing off-topic websites or software, checking email or phones, *utilizing large language models* or otherwise attending to things not pertinant to the course is distracting to yourself and others and will result in point deductions.
- Embrace intellectual humility. Recognize that there are limitations to your knowledge and that some of your beliefs could actually be wrong. Be curious about your thinking and open to learning from others. This is hard, but so vital to your learning in this course—we'll work on developing this throughout the term!
- Get help when you need it. If you are stuck or confused or lost, be proactive and get help. The best learners and thinkers can figure out when they are stuck and make a plan to get unstuck. Come to Office Hours (Steinmetz 108B), the CS Helpdesk (Sun-Thurs 7:00-9:00pm Olin 107) or ask another student. Often the best person to explain something is the person who just figured it out. Try reaching out to another student in the course who seems like they get it—they will probably be flattered you asked!

15 ACADEMIC INTEGRITY AND "ARTIFICAL INTELLIGENCE" USE

Union College recognizes the need to create an environment of mutual trust as part of its educational mission. Responsible participation in an academic community requires respect for and acknowledgement of the thoughts and work of others, whether expressed in the present or in some distant time and place. Matriculation at the College is taken to signify implicit agreement with the Academic Honor Code, available at: http://muse.union.edu/honorcode

It is each student's responsibility to ensure that submitted work is their own and does not involve any form of academic misconduct. Students are expected to ask their course instructors for clarification regarding, but not limited to, collaboration, citations, and plagiarism. Ignorance is not an excuse for breaching academic integrity. Students are also required to affix the full Honor Code Affirmation, or the following shortened version, on each item of coursework submitted for grading:

'I affirm that I have carried out my academic endeavors with full academic honesty.'

[Signed, Jane /John /Jaden Doe]

What it means for this course: **you must complete the programming projects on your own, but you are welcome and encouraged to collaborate on other regular assignments.**

Please notice that different rules apply to the exercises that are part of the textbook, regular assignments and programming projects.

Textbook Readings and Regular Exercises:

You are welcome to discuss the textbook readings, the exercises embedded in the textbook, and other regular assignments with other people in the class to help you fully understand the material. However, don't just copy somebody else's solution. Even if you ask other people for help, the goal is for you to understand the underlying concepts and logic. It's more important to understand how to arrive at a correct solution than it is to know the solution itself.

Programming Projects:

Any work you turn in has to be your own. The best way to learn programming is by doing programming. I expect that you already know that if you do not do things for yourself, you will not learn them. Sometimes that will mean that you will have to struggle. That is ok: struggling to figure out how to solve a problem is where a lot of learning happens. Give yourself the opportunity to do this.

Please, do not ever copy any part of a solution from a friend, from any material generated by an "artificial intellence" (AI) chatbot model (ChatGPT, Gemini, Copilot, Claude, etc.) or from anywhere else. Do not ever *look at* somebody else's solution or any solution generated by an entity other than yourself before you have completed your own. Do not ever *show* somebody else your solution before you've both submitted it; though it may be beneficial to discuss your solution with another student in the class after you've both handed in the assignment. Please do not *ever* show your solution to future students of this class or any chatbot, and do not *ever* post your solutions on the Internet.

Don't search for information on the Internet. Don't ask for information from any AI model. Especially, don't look at any material that provides solutions to exercises that are similar to the projects. An exception is, of course, any online material that is linked directly from the Nexus page. As always, you should practice good citation behavior and cite any sources that you consult in your work (for example, which pages from the Python documentation website you consulted when working on the project).

Discussing your work (without looking at code): Talking through a problem is extremely beneficial in understanding the problem and coming up with a solution path. So, I strongly encourage you to talk to other students in this class about the projects, visit the CS Helpdesk and come to my Office Hours (you can also request a meeting if you have schedule conflicts with my Office Hours).

Here are some ways in which you can talk about the projects without having to worry about violating the rules from above.

- It's ok (and strongly encouraged!) to draw **memory diagrams** together. Draw memory diagrams and discuss possible algorithms for solving the problem.

- It's ok to **write down an algorithm in English** together. Then go off by yourself to implement it into code.
- It's ok to read and write code together that is not part of the assignment, but that helps demonstrate the concept of what you're being asked to do. Go through an example from class or from the book together, or **come up with your own examples**. There's no better way to understand something than trying to think up examples in order to teach it to someone else. Try it!

Do not work on the implementation together with friends, especially if you are each at your own computer and are always in lock step trying to figure out the same line or section of code at the same time. Have a higher-level discussion (using one of the strategies suggested above), then work on the implementation on your own.

Your goal for discussions about any assignment should be that you come away with a better understanding of the problem and of possible ways to approach it so that you can then try out these approaches on your own. You should never leave a discussion with just an answer, without an understanding of how to arrive at that answer. A good general guideline for any discussion, or interaction with an AI model, is that you should not leave the discussion with anything written or typed.

16 CONTACTING ME OUTSIDE OF CLASS

The best method for contacting me outside of class is to stop by my office during Office Hours or whenever my office door is open. You can also schedule a time with me if my regular office hours don't work. Please contact me via email (doingg@union.edu) and we can arrange a phone or zoom call. I respond to emails as soon as possible, but it may take up to one day to get a response during the term, and longer between terms.

17 ADDITIONAL RESOURCES

Mental Health and Campus Resources

Union College is committed to supporting and advancing the mental health and well-being of our students. During the course of their academic careers, students often experience personal challenges that contribute to barriers in learning, such as drug/alcohol problems, strained relationships, chronic worrying, persistent sadness or loss of interest in enjoyable activities, family conflict, grief and loss, domestic violence, difficulty concentrating, problems with organization, procrastination and/or lack of motivation. Students also sometimes come to college with a history of learning difficulties (e.g., any form of special education), experience difficulties succeeding in a particular subject (e.g., math, reading), or have experienced some form of trauma be it emotional or physical (e.g., head injury). These mental health concerns

can lead to diminished academic performance and can interfere with daily life activities. If you or someone you know has a history of mental health concerns or if you are unsure and would like a consultation, a variety of confidential services are available. The Eppler-Wolff Counseling Center provides free counseling and therapy services (including psychiatry) to all enrolled students. Please call (518) 388-6161 or visit the Wicker Wellness Center in person any weekday between 8:30 and 5:00 to schedule an initial contact appointment. Visit the Counseling Center website for more information.

In a crisis situation, or after hours, contact Campus Safety at (518) 388-6911. The National Suicide Prevention hotline also offers a 24-hour hotline at (800) 273-8255.

Any student who faces challenges securing their food or housing and believes this may affect their performance in the course is urged to contact the Dean of Students (dos_office@union.edu) for support or drop into office hours Monday - Friday 8:30 am - 5:00 pm in the Reamer Campus Center room 306. Furthermore, please notify me if you are confortable doing so and I will provide any resources I can. The Union College Persistance Fund can provide financial support in times of unexpected harship to cover needs such as emergency medical expenses not covered by insurance and basic living expenses. Additional sources of supoprt are outlines on the Dean of Students webpage: https://www.union.edu/dean-students .

18 TENTATIVE SCHEDULE

W	TOPIC	Due by end of week	Runestone
1	Programming, Syntax and Semantics	Portfolio (PF) 1	1 & 2
2	Variables and Functions	PF 2	3 & 4
3	Conditional Statements	PF 3, Project Demo 1	5
4	Conditional Loops	PF 4, Practical Exam 1	6
5	Counting Loops	PF 5, Project Demo 2	7
6	Strings, Lists	PF 6, Written Exam 1	8 & 9
7	Nested Lists, Dictionaries	PF 7, Project Demo 3	10
8	File I/O	PF 8, Practical Exam 2	11
9	Recursion	PF 9, Project Demo 4	
10	Searching, Sorting	PF 10	
	Finals Week Monday 11/23 2:30 - 4:30	Written Exam 2	

18.1 ADDENDA

All updated text in this syllabus will supersede text from older versions.

19 PROGRESS TRACKER

Learning Goals	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
What are the most important programming fundamentals and why are they the building blocks of programs?										
What are the debugging fundamentals and how do you use them?										
What are some of the mechanisms used to build “intelligent” machines?										
Use variables, functions, conditionals, loops and recursion.										
Write short Python programs that solve a given problem.										
Fix short Python programs using debugging fundamentals.										

Grading

Table 2: Grading Scheme

Course Element	Grade Point Contribution	Notes
Reading & Practice	5%	Runestone Interactive Textbook
Portfolios & Share-outs	15%	From in-class and take-home problems
Programming Project Demos	20%	4 at 5% each
Practical Exams	20%	2 at 10% each
Midterm	20%	
Final Exam	20%	written
total	100%	

Table 3: Letter Grade Scale

Letter Grade	Percent range
A	93 - 100
A-	90 - 92
B+	87 - 89
B	83 - 86
B-	80 - 82
C+	77 - 79

Letter Grade	Percent range
C	73 – 76
C-	70 – 72
D	60 – 69
F	0 – 59

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Assignments

PF 01

CSC106: Portfolio 1

- due Wednesday 9/16/26 by class time
- Intro Survey
- Runestone Chapters 1 & 2
  - https://runestone.academy/runestone/default/user/register
  - your Union College email
  - unioncollege_foppff_fall26 as the course name
- Do your share-out
  - check the schedule to see what problem and what date you will share:
  - add your solution to the share-out slide-deck
- Finish all parts of the Week 1 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
  - Starter code: winding_sandbar.json

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Wed. Sept. 16th by the end of class.

Portfolio 1: Programming Syntax and Semantics

To succeed in these problems you will need to read and engage with the embedded activities in the following **Runestone texbook chapters** (worth 0.5 of a grade percentage point):

- Ch. 1: Introduction
- Ch. 2: Variables, Statements and Expressions

For completing and self-correcting 70% of the below activities (during in-class share-outs), you will earn 1 grade percentage point. For participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own ("oyo"). You should finish each activity before its in-class share-out in order to correct it before final submission. Note the intended schedule below but keep in mind that the schedule is always subject to change.

Remember to print and attach your programs for the coding activities. Programs should be submitted to gradescope for autograder results, but credit is given to programs printed and included in this portfolio, on completetion, regaurdless of autograder results. Programs should be marked up for corrections during in-class share-outs.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt: think?	2	W 9/9	W 9/9	Hand-written
A2	Written programs for human turtles	3	W 9/9	W 9/9	Hand-written
A3	Written questions - computer basics	5	Th 9/10	Th 9/10	Hand-written
A4	Prog. with Reeborg (patterns)	7	Th 9/10	Th 9/10	Printed
A5	Prog. with Reeborg (newspaper)	9	Th 9/10	F 9/11	Gradescope & printed
A6	Questions - syntax, semantics, prompt	11	F 9/11	M 9/14	Hand-written
A7	Tracing Re-assignment	13	oyo	M 9/14	Hand-written
A8	Prog. Arithmetic, reciept	15	M 9/14	W 9/16	Gradescope & printed
A9	Prog. Reeborg (winding sandbar)	17	oyo	W 9/16	Gradescope & printed
A10	Prog. with Turtle (squares)	19	oyo	W 9/16	Gradescope & printed
B1	Share-out report	21	oyo		Hand-written

A1) Prompt: Can computers think? Why or why not? What does it mean, to "think"?

A2) Programs for Human Turtles

1. Fold this sheet into thirds & turn it over.
2. On the back top third, draw something that could have been drawn by the Turtle robot.
3. On the front, in the top two thirds, write instructions (in English) for the robot to create this drawing.
4. Swap your instructions with someone who didn't see your drawing.
5. **Without looking at the drawing on the back** draw a picture following the instructions.
6. Fold down the top third and see - how closely does your drawing match the original?

……

instructions:

……

leave blank for drawing:

……

Draw something a turtle-bot could draw:

……

A3) Computers, Algorithms and Programs

- Take a look at **Runestone Chapters 1 & 2** … feel free to look at it while answering these questions and during today's discussion

0. Name some parts of a computer.

   ……

   ……

   ……

   ……

1. Where are the programs you write stored when the computer is powered off? Why?

   ……

   ……

   ……

   ……

2. Where are the programs currently running on a computer stored? Why?

   ……

   ……

   ……

   ……

3. What is a reserved word in Python? Name a few.

   ……

   ……

   ……

   ……

4. How would you describe the difference between an interpreted programming language and a compiled programming language?

   ……

   ……

   ……

   ……

5. Describe some pros and cons of interpreted and compiled programming languages.

   .....

   .....

   .....

   .....

6. Can more than one algorithm be used to solve a problem? Why or why not?

   .....

   .....

   .....

   .....

7. Can an algorithm be written in more than one programming language? Why or why not?

   .....

   .....

   .....

   .....

8. What is a function? Why might functions be useful?

   .....

   .....

   .....

   .....

9. What is a parameter?

   .....

   .....

   .....

   .....

10. What is an argument?

    .....

    .....

    .....

    .....

A4) Programs for Robots: Working with Reeborg

Paired Programming Instructions

- Pair programming is a widely utilized programming method popular in industry and academia for the increase in quality it gives to both process (learning, comprehension, memory) and product (algorithm, program, results).

Paired programming guidelines:

- Roles:
  - driver: has the keyboard and types
  - navigator: guides the driver
- Instructions:
  - You should be talking to each other constantly!
  - You should be able to swap roles at any moment.
  - Swap roles every 5 to 10 minutes.

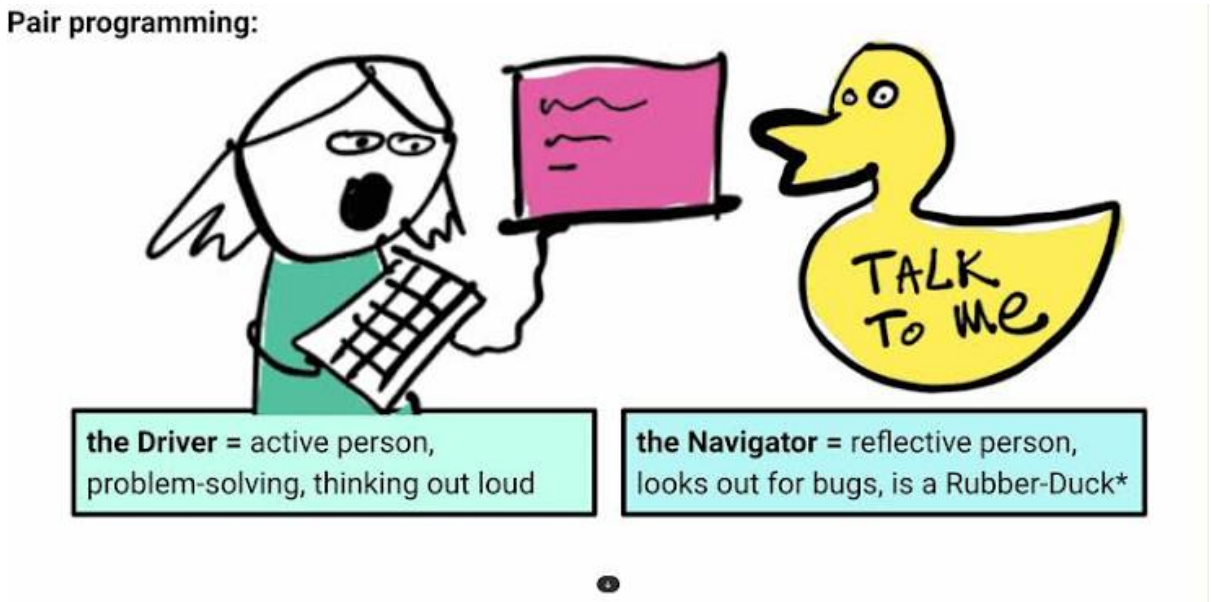
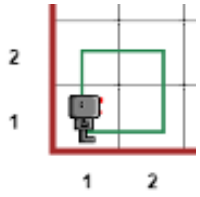


Figure 1: Pair Programming

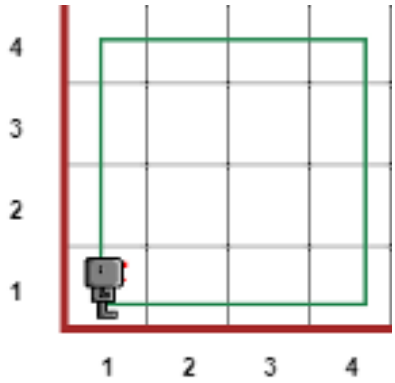
- Note that most programming we do in class will be pair programming. The exceptions will be in-class practical exams where each student will work independently.
- You will have the opportunity to practice indeipendent programming when finishing up assignments you don't finish in class, doing on assignments we don't start in class and doing your projects.

From the course website, under Resources, follow the link to Reeborg's World, or type out the url: https://reeborg.ca/reeborg.html

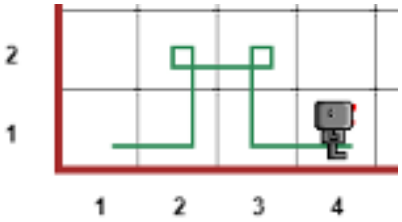
1. Write programs to make Reeborg walk the following patterns. Save each program in separate files.



(a) Reeborg_pattern1



(a) Reeborg_pattern2



(a) Reeborg_pattern3

Reeborg patterns

2. In english (not code), what is your algorithm for each pattern?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

- To save, choose "Save program to file" from the "Program in editor" section of the "Additional options" menu.
- Note: Python programs are saved as text files, but the file names have the extension `.py`

3. Upload each file to your Google drive, or email to yourself, so you can print out these files to submit with your portfolio.

For portfolio submission, insert you printed program pages here.

A5) Programs for Robots: Delivering a newspaper

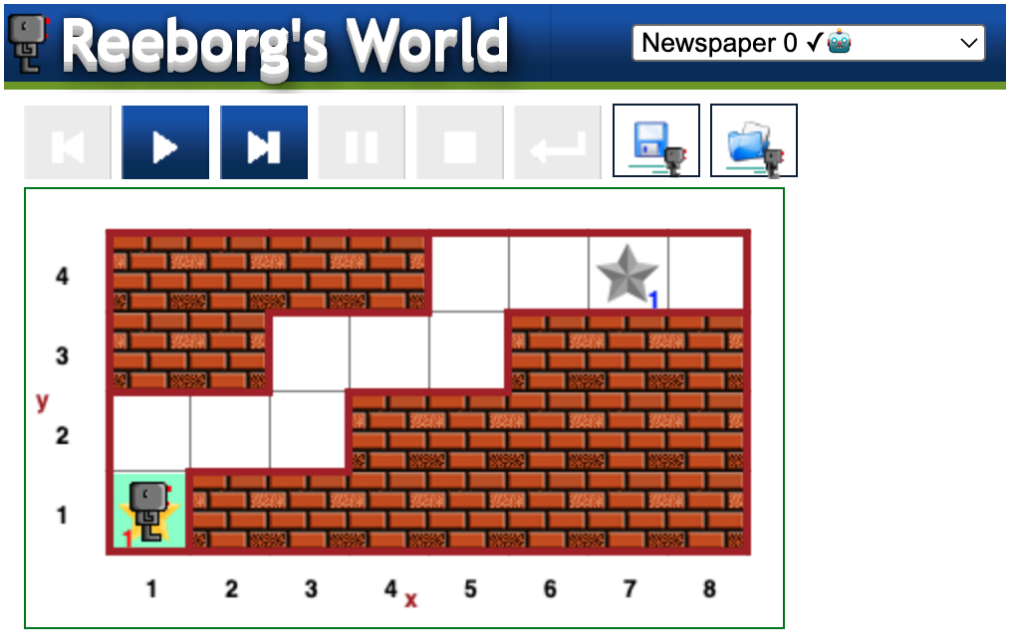


Figure 5: Reeborg Newspaper 0

1. Select the `Newspaper 0` world. It should look like the image above. Find the "World Info" button, just to the right of the upper right hand corner of the above screenshot. What is the goal in this world?

   .....

   .....

   .....

   .....

2. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

3. Write a program to implement your algorithm and solve `Newspaper 0`.
4. Add comments to your program to explain what it's doing (each step of your algorithm).
5. Save the program in a file called `newspaper0.py`

6. Submit `newspaper0.py` on gradescope:

    a. Follow link to gradescope (link can be found on the course website)
    b. Select Union College from the list of options
    c. Login using your Union College credentials
    d. Find the assignment: `In-class Assignment: Newspaper 0`
    e. Upload `newspaper0.py` to the assignment

7. Did the autograder produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

8. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

9. Share the code you worked on with your partner(s) before you leave class. Upload it to your Google drive or email it so you can each complete it, if not already done, and print it out.

For portfolio submission, insert you printed program pages here.

A6) Check-in Questions, Syntax and Semantics

Answer the following questions about Reeborg and Python (Turtle) syntax. Think about what the program syntax looks like and means. Programs can be valid, meaning they have proper syntax, but to be *correct* they must also do what the programmer intends them to do, meaning the semantics are right.

Take a look at Runestone Chapter 2 before completing the rest of this portfolio.

1. Which of the following programs are valid Reeborg programs that cause Reborg to execute a sequence of actions? What's wrong with the invalid ones? (Assume Reeborg won't run into walls)

   a.
   ```
   Move()
   Move()
   Move()
   ```

   b.
   ```
   move()
   turn_left()
   turn_left()
   turn_left()
   move()
   ```

   c.
   ```
   move()
   move()
   turn_right()
   move()
   ```

   d.
   ```
   move
   turn_left
   move
   move
   ```

2. The following is a function call statement:
   `turn_left()`

   a. What is the name of the function being called?

   .....

   b. What purpose do the parentheses serve?

   .....

   c. What is a function?

   .....

   .....

   d. What does "calling a function" mean?

   .....

   .....

3. Consider the following Python program. Draw the pattern that the turtle draws when this program is executed. Also indicate the position and orientation of the turtle after this program has been executed. Hint: The turtle starts out in the center of the turtle window, facing to the right

```
import turtle

turtle.left(90)
turtle.forward(200)
turtle.left(180)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
```

4. Which of the following programs are valid Python programs? Indicate which are valid. For the invalid ones, explain or indicate what is wrong. The import statement isn't shown, but you can assume that the turtle module has been imported.

   a.
   ```
   turtle.forward(100)
   turtle.right(90)
   turtle.forward(100)
   ```

   b.
   ```
   turtle.forward
   turtle.left
   turtle.forward
   ```

   c.
   ```
   turtle.forward(100)
   turtle.turn_right(90)
   turtle.forward(100)
   ```

   d.
   ```
   turtle.Forward(100)
   turtle.right(90)
   turtle.Forward(100)
   ```

5. What required the most thinking — coming up with algorithms, writing programs or communicating in pair programming? Did the activities rquire different *types* and/or *amounts* of thinking?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A7) Tracing (re-)assignment with Memory Diagrams

Consider the following snippet of Python code.

```
1.  x = 5
2.  y = 10
3.  z = x
4.  x = y + 3
5.  y = z
```

1. Draw a memory diagram that shows the <variable name>-<object/value> associations after line 3 has been executed.

2. Draw a memory diagram that shows the <variable name>-<object/value> associations after line 4 has been executed.

3. Draw a memory diagram that shows the <variable name>-<object/value> associations after the whole snippet has been executed.

4. In Python, what happens in the machine's memory blocks when a variable is assigned to another variable? Do the variables share a memory block, or is a new memory location used? Assume the inital variable was assigned to an integer value.

5. Depict the process you described above with a diagram/drawing.

A8) Arithmetic in Python

Computer programming languages are often designed to handle arithmetic operations such as addition, subtraction, multiplication, division, and exponents. These operations are performed through the use of operators, several of which are listed below:

$$+, -, *, /, //, \%, **$$

Just as in mathematics, we can use these operators to develop complex mathematical equations. Also like in math in a more general sense, parentheses can be used to give some operations a higher priority than they would normally have. For example:

$$2 + 4 * 2 = 2 + 8 = 10$$
$$(2 + 4) * 2 = 6 * 2 = 12$$

1. Using IDLE, write a Python program that uses each of the operators shown above at least once. Please include comments explaining what it looks like each operator does. You may need to experiment with some of them to figure out exactly what they're doing. Name your program `arithmetic.py`.
2. Add print statements to give the same information as the comments so a user will see an explanation of what the program is doing whenever they run it.
3. Taxes, tips, and totals: Now we're going to write a program to calculate the tax, tip, and total cost for dining at a restaurant. Name your program `receipt.py`.

   Start by assigning the following values to variables:

   i. The meal's subtotal of $35 (example name: `meal_cost`)
   ii. The meal's tax rate of 7% (example name: `tax_rate`)
   iii. The meal's tip rate of 25% (example name: `tip_rate`)

4. Add code to compute the tax, tip, and total cost for the meal, then use print() to display a receipt that displays the following information:

```
RECEIPT
Sub-total: $35
Tax:       $2.45
Tip:       $8.75
Total:     $46.20
```

5. Make the numbers line up nicely. For the time being, it's perfectly acceptable if $46.20 appears as $46.2 instead.
6. Change the values assigned to each of the three variables you created. Do the results produced by the altered program make sense? (e.g. is the tip computed correctly?) If not, then figure out why and try to fix it.
7. Adjust the program so that `meal_cost`, `tax_rate`, and `tip_rate` are assigned from user input (i.e. using the `input()` function and type-casting).

8. Test the new version of the program to make sure that the receipt information is correct based on the input.
9. If you haven't already done so, make sure to add comments explaining how the program works.
10. Can you display the whole receipt using a single `print()` function call?
11. Did the autograder produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

12. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

For portfolio submission, insert you printed program pages here.

A9) Programming practice: Reeborg

0. Do this part on the Reeborg's World website.
1. Download the starter code from the course website.
    - It is one file called: `winding_sandbar.json`
2. Import the starter file winding_sandbar.json into Reeborg.
    - To import a program, open the "Additional options" menu.
    - Then look for the button "Open world from file."
    - This should show you an environment that looks as follows:

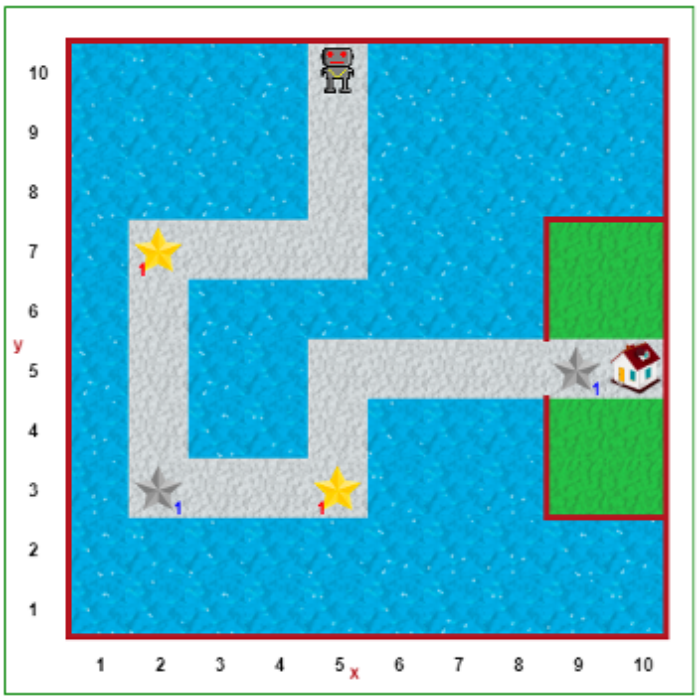


Figure 6: Reeborg Winding Sandbar

3. What is the goal in this world?

.....

.....

.....

.....

.....

4. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

5. Complete the program so that Reeborg picks up the stars, deposits them in the indicated spots and finishes at the house. Save your program file (not the world file) in a file with the name `winding_sandbar.py`.

   - When pair-programming, as you go, add comments throughout indicating who was the "driver" and who was the "navigator" for each chunk.

6. Share the program between pairs using email or Google drive/
7. Individually add comments to your code. Explain the different steps and including a header comment that names your partner.
8. Submit your code on gradescope. Remember to also print it out to submit with your portfolio.
9. Did the autograder produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

10. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

For portfolio submission, insert you printed program pages here.

A10) Programming Practice: Turtle in Python

0. Your goal is to write a program that uses the turtle module to draw the following picture.

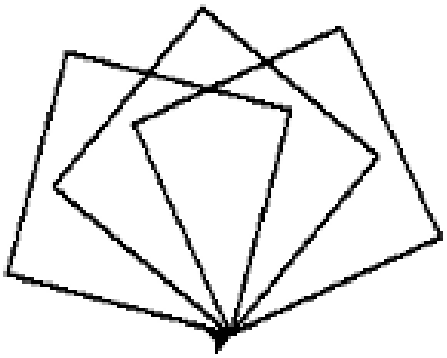


Figure 7: Turtle Squares

1. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

2. Now, in IDLE, write your program in Python. Save your program in a file called squares.py
3. Think about how you would modify your program so that it draws the same three squares, but each of them in a different color.

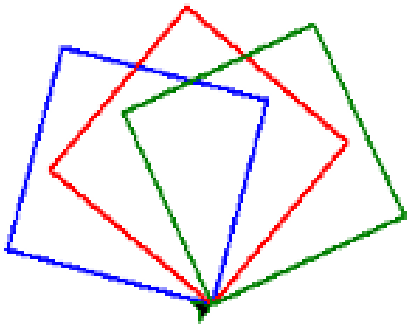


Figure 8: Turtle Squares in Color

3. (cont.) In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

4. Now edit your previous program to implement your new algorithm. Don't forget to add comments to your code to help human readers. Save your program in a file called colored_squares.py.
5. Submit your code on gradescope. Remember to also print it out to submit with your portfolio.
6. Did the autograder produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

7. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

For portfolio submission, insert you printed program pages here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out 2 (optional, for make-ups)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

PF 02

CSC106: Portfolio 2

- due Friday 9/25/26 by class time
- Intro Survey
- Runestone Chapters 3 & 4
  - https://runestone.academy/runestone/default/user/register
  - your Union College email
  - unioncollege_foppff_fall26 as the course name
- Do your share-out
  - new schedule
    - open spots can be claimed ro re-arrange or make-up
  - new slides
- Finish all parts of the Week 2 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
- stars0.json
- stars1.json
- house.py

- birthday.py
- cricket.py

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Fri Sept. 25th at the end of class.

Portfolio 2: Variables and Functions

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapters** (worth 0.5 of a grade percentage point):

- 3: Debugging
- 4: Functions

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point.*

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Remember to print and attach your programs for the coding activities. Programs should be submitted to gradescope for autograder results, but credit is giving to programs printed and included in this portfolio, on completetion, reguardless of autograder results. Programs should be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Prompt: randomness	2			written
A2	Functioncs for Reeborg (stars, carrots)	3	Th 9/17	M 9/21	GS, print
A3	Functions for Turtle (flowers)	5	F 9/18	M 9/21	GS, print
A4	Functions I/O (even, math quiz, return)	7	M 9/21	W 9/23	GS, print
A5	Parameters for Turtle (house)	11	W 9/23	Th 9/24	GS, print
A6	Parameters in Python (ascii art)	13	W 9/23	Th 9/24	GS, print
A7	Study questions	15	oyo	Th 9/24	written
A8	Prog. Python (temp, bookstore)	18	oyo	Th 9/24	GS, print
A9	Prog. Python (birthday)	21	oyo	Th 9/24	GS, print
A10	Prog. Python (cricut)	23	oyo	Th 9/24	GS, print
B1	Share-out report	25	oyo	n/a	written

A1) Prompt: Is randomly generating values creative? Does it produce novel information? Is it "just" following instructions? Is it thinking? Why or why not??

A2) Writing Functions for Reeborg

Writing programs with functions

Going forward, you should be looking for opportunities to define functions to make your code less repetitive and easier to read.

In general, a function should:

- handle one clearly defined logical sub-task (i.e. it shouldn't be your whole program)
- have a name that describes (or at least hints at) the function's purpose
- be accompanied by a comment that describes what the function does in more detail than what can be communicated solely through the function name
  - (very optional) While completely optional in this course, if you plan to go into software development, it may be useful to look at the page for Python's Docstring Conventions (https://peps.python.org/pep-0257/) to see a common way functions are documented in professional settings. Strictly speaking docstrings are slightly different from traditional comments, but they will be counted as such when used to describe a function in this course.

Reeborg Picking Up Stars

1. Download the starter file `stars0.json` from the course website and import it into Reeborg's World.
2. Define a function `collect_row_of_stars` that helps Reeborg pick up all the stars. When you call this function, Reeborg should move across the world and collect all the stars lined up on the bottom row. Use loops as appropriate. You can also define other functions.
3. Download the starter file `stars1.json` from the course website and import it into Reeborg's World.
4. Write a program that helps Reeborg pick up all the stars in the world. Can you reuse any function(s) that you defined to solve the previous world? First, in english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....
5. Don't forget to include comments in your code.
6. Name your program `picking_up_stars.py` and submit to Gradescope.

Reeborg Harvesting Carrots

7. Open the tutorial world Harvest 1.
8. Write a program that helps Reeborg harvest the field of carrots.
    - Before you start writing code, think about different approaches (i.e. walking paths) that Reeborg could use to keep things efficient.
    - Try to choose a path that allows you to make good use of function definitions and loops to create a program that's easy to read.
    - First, in english (not code), what is your algorithm for your solution?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....
9. Don't forget to include comments in your code.
10. Name your program `harvesting_carrots.py` and submit to Gradescope.
11. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....
12. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

A3) Functions for Turtle

Turtle: Flower Meadow

1. Define a function called `flower` that uses the `turtle` module to draw a flower (any colors):



Figure 1: Flower

2. Use the flower function to draw a flower meadow like in the picture below (use whatever colors you like, but use at least three different colors). You can set the background color using the following function: `turtle.bgcolor(<specified color>)`

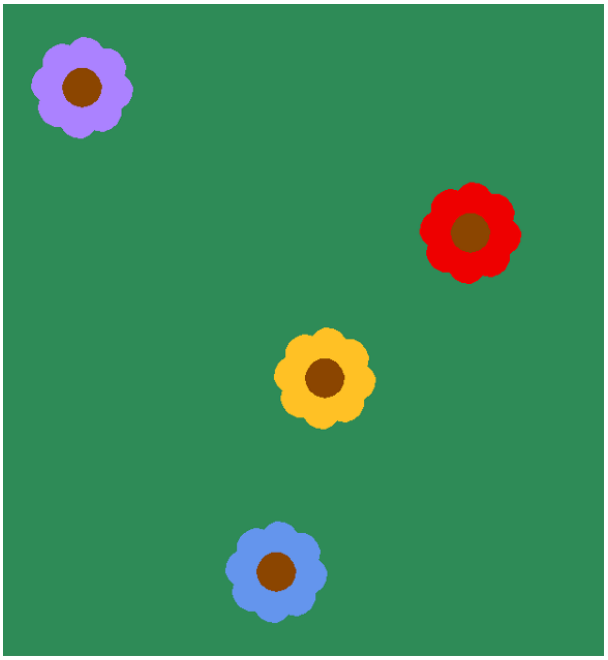


Figure 2: Flowers

3. (Optional) Try to figure out which colors were used in the example pictures. All the colors used can be found on this web page: http://cs111.wellesley.edu/archive/cs111_fall14/public_html/labs/lab12/tkintercolor.html

   .....

   .....

   .....

4. First, in english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

5. Don't forget to include comments in your code.

6. (Optional) Modify your program so that the locations of the flowers are random. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

7. Name your program `flower_meadow.py` and submit to Gradescope.

8. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

9. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

A4) Functions: Input/Output

Practicing functions with user input: Is it even?

1. Define a function called `is_even` that determines whether a given integer is even.

- The user inputs a number
- The program checks whether the number is even or odd and prints the result

Remember Python's arithmetic operators:

$$+, -, *, /, //, \%, **$$

2. What does it mean for a number to be even? Which of the operators will help you test for evenness? In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

3. Write your function in a program named `math_io_functions.py` , adding comments.

Math facts quiz

4. Write a function `math_quiz` that helps elementary students practice their math facts.

- Ask the user to input two numbers
- Ask the user which number is greater
- If the user's answer is right, print :

```
"Correct!"
```

- If the user's answer is wrong, print:

```
"Incorrect, the correct answer is <the correct value>!"
```

5. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

6. (Optional) Add the functionality to randomly generate the two numbers instead of user input. Present the numbers to the user, and then wait for the user to input a result.
7. (Optional) If the user's result is correct, the function should print out a message indicating as much. Otherwise, it should say that the answer was incorrect and show the right answer.

Remember to give your programs meaningful names (ending in `.py`) and submit to Gradescope.

8. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

9. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

Defining functions with return values: User input and output vs. function input and output

- user input/output: to communicate with the *user* of a program.
  - **user input**: built-in function `input()`
  - **user output**: built-in function `print()`
- function input/output: for different parts of a program to communicate with each other. e.g. between two functions, between the global namespace (scope) and a function's namespace (scope)
  - **function input**: the function's parameters
  - **function output**: the function's return value

Functions that take parameters and return values are more versatile than ones that directly ask a user for input and respond accordingly. Functions that take parameters and return results can still be used in programs that ask for user input then call the function with the user input as parameter values.

In contrast to functions that take parameters and return values, functions that rely on direct user input are more limited in their applicability. Specifically, these functions can't be used as building blocks for larger programs where the function needs to be able to handle values calculated by other parts of the program or where the function's results need to be processed further.

Please be sure to read problem specifications carefully. Unless the instructions specify otherwise, generally, you should assume that any input values the function requires should be provided as parameters and that any results the function calculates should be returned.

10. Circle area - Define a function called `circle_area` that calculates the area of a circle. Given the circle's radius, the function should return the area.

    - The area of a circle is given by the following equation:

    $$Area = \pi r^2$$

    - Python's math module contains a variable called pi that gives you an approximation of π. Given what you know about modules, how can you use this predefined variable in your code?
    - Once you have imported the math module (`import math`), you can access π with: `math.pi`
    - Note the lack of parentheses. Pi, here, is a variable, not a function, so no parentheses are used.
    - In english (not code), what is your algorithm for your solution?

      .....

      .....

      .....

      .....

      .....

      .....

      .....

11. Pizza cost per square inch - Define a function called `pizza_cost` that calculates then returns the cost per square inch for a circular pizza, given the pizza's diameter and total price.

    - You should use the `circle_area` function from the previous question to help calculate the expected value.
    - In english (not code), what is your algorithm for your solution?

      .....

      .....

      .....

      .....

      .....

      .....

      .....

12. Coffee intake - Define a function that calculates how much coffee will fit into a cylindrical mug, given its radius and height. You should be able to reuse the `circle_area` function you've previously defined.

    - The volume of a cylinder is given by:

    $$V_{cylinder} = A_{circle} * height$$

    - In english (not code), what is your algorithm for your solution?

    .....

    .....

    .....

    .....

    .....

    .....

13. Random Sizes - Test your functions by using the `random` module to generate random sizes for the radius of a circle, cost of a pizza, and height of a mug.

14. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....

    .....

15. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

A5) Functions for Drawing: St. Nick's House

1. St. Nicholas' house

1. Download the starter code `house.py` from the course website. The file contains code to draw a house-like shape .
    - German kids know this shape as "St. Nicholas' house". It can be used as a puzzle where you have to draw the shape without lifting your pencil and without tracing.
    - For example:



Figure 3: St. Nick's Houses

2. Define a function called `house` that draws the shape.
3. Define a function `fly_to` that does the same things as the `turtle.setpos` / `turtle.goto`, but doesn't leave a line behind as the turtle moves.
4. Use these functions to draw a few houses in different locations
5. Modify the `house` function so that it allows you to draw houses of different sizes and colors. Start by figuring out how many parameters your function needs and what each parameter represents.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

6. Write a program, in the `house.py` file, to draw a picture of houses in different colors and sizes.
7. Modify your program so that the size and location of the houses is random.
8. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

9. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

A6) ASCII art

ASCII art uses keyboard characters to create pictures. Name your file `ascii_art.py`. For example:

```
Zzzzz  |\      _,,,---,,_
     /,`.-'`'    -.  ;-;;,_
    |,4-  ) )-,_..;\ (  `'-'
     '---''(_/--'  `-'\_)
```

1. Define a function called ascii_tree that uses the print function to display the following picture:

```
   *
  ***
 *****
*******
  ***
```

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

2. Now add two parameters to the function which specify which characters to use for the branches and the trunk. The function should now be able to print the following:

```
   o
  ooo
 ooooo
ooooooo
  nnn

   -
  ---
 -----
-------
  ###

   ~
```

```
  ~~~
 ~~~~~
~~~~~~~
  vvv

   i
  iii
 iiiii
iiiiiii
  uuu
```

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

3. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

4. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A7: Written Questions

1. 1. Which of the following are valid first lines for a Python function definition? Please explain why the invalid ones are not valid.

a. `def plus ten (anumber):`

b. `def double (anumber)`

c. `def greeting:`

d. `percentage(total, part):`

e. `def print_introduction():`

f. `def move()`

g. `def goto(x coordinate y coordinate):`

2. Print or return?

Consider the following (incomplete) function definitions.

Given the way these functions are used in the code at the end, complete the dotted lines by either turning them into a statement that **returns** the indicated value or into a statement that **prints** the indicated value.

Also complete the dotted lines in the code at the end by choosing to either `return` or `print` the value of the variable `a`.

```
def f1():
    a = int(input('Please input a number: '))

    ......\textcolor{red}{return}.....a........

def f2(var):
        squared = var**2

    ..................squared........

def f3(var):

    ..................var**3........

a = 10

..................a........
f2(a)
print(f1() + 3)
a = f3(a) * 5

..................a........
```

f. Assuming that the user input 1 when asked for a number, what will this code print to the screen?

.....

.....

.....

.....

.....

.....

A8:

Function parameters vs. user input

Consider the following two programs:

```
1)  # Temperature Conversion Version 1
2)  def fahrenheit_to_celsius(fahrenheit):
3)       celsius = (fahrenheit - 32) * 5 / 9
4)       print(fahrenheit, "deg Fahrenheit is", end=" ")
5)       print(celsius, "deg Celsius.")
6)
7)  temperature = int(input("What is the temperature in Fahrenheit? "))
8)  fahrenheit_to_celsius(temperature)
```

```
1.  # Temperature Conversion Version 2
2.  def fahrenheit_to_celsius():
3.       fahrenheit = int(input("What is the temperature in Fahrenheit? "))
4.       celsius = (fahrenheit - 32) * 5 / 9
5.       print(fahrenheit, "deg Fahrenheit is", end=" ")
6.       print(celsius, "deg Celsius.")
7.
8.  fahrenheit_to_celsius()
```

1. From the user's point of view, do these programs behave differently? .....

.....

.....

.....

.....

.....

2. From the programmer's point of view, which one is better and why?

.....

.....

.....

.....

.....

.....

3. Bookstore - Define a function called `book_order`, which should take the cover price of a book and the number of copies the store wants to order as its parameters. Suppose that bookstores get a 40% discount. Shipping costs $3 for the first copy and $0.75 for each additional copy.

- The function should print out a message indicating
  - the cover price of the bok
  - the number of copies ordered
  - what the bookstore will have to pay for the books (don't forget the discount)
  - the shipping cost
  - the total amount the bookstore will have to pay (for books + shipping)
- For example, if you called `book_order(12.97, 15)` (meaning the store orders 15 copies of a book that costs $12.97), it should print:

```
cover price: $12.97
copies ordered: 15
order cost: $116.73
shipping: $13.50
total: $130.23
```

- Hint: you can use the built-in round function to `round(<value>, <decimal places>)` to a desired number of digits after (or before) the decimal point. You can find the documentation in the Python Library Reference section on Built-in Functions, which can also be found through the Resources section on the course website: https://docs.python.org/3/library/functions.html
- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....
- Don't forget to comment your code and organize it in a sensible way.
- Don't forget to include your answers from section 3 as comments in the code.
- Save your program in a file like `bookstore.py`.

A9: Happy Birthday

1. Download the starter file `birthday.py` from the course website and read the instructions inside before continuing.
2. Write a function that "sings" Happy Birthday for someone. The function should take the person's name as a parameter and then print Happy Birthday in the following format.:

```
Happy Birthday to you!
Happy Birthday to you!
Happy Birthday dear <name>!
Happy Birthday to you!
```

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

3. Make sure to include appropriate comments.
4. Save your program using the filename `birthday.py`.
5. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

6. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

.....

A10: Cricket Chirping

1. Download the starter file `cricket.py` from the course website and read the instructions inside before continuing.
2. Amos Dolbaer discovered that, for certain types of crickets, there's a relationship between the temperature and the rate at which they chirp (https://en.wikipedia.org/wiki/Dolbear's_law). More specifically, the relationship is as follows:

$$T = 50 + (N - 40)/4$$

where T is the temperature in Farenheight and N the number of chirps per minute.

Write a function that calculates the temperature when given the number of times a cricket chirps in 15 seconds. Your function should do the following:

a. calculate the chirps per minute
b. calculate the temperature based on chirps per minute
c. return the temperature

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

.....

3. Save your program in a file called `cricket.py`
4. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

5. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

.....

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out 2 (optional, for make-ups)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

Project 1

CSC106: Project 1

- due Monday 9/28 ~~Wednesday 9/23/26 by class time (1:50 pm)~~
- demo instructions at the end of the PDF description
  - demo time slots
  - move yourself to an empty slot if you need to
  - demos in Steinmetz 209A
- you may use late token(s) for this assignment
  - email me to arrange the demo in this case

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting, and demo performance
- This project is 5% of your final grade
- starter code:
  - square.py

  - turtle_shapes.py
  - computer_art.py

- Demo instructions

NAME:

Project 1: Drawings with Turtle

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

Learning Objectives:

1. Practice designing and implementing simple algorithms.
2. Demonstrate that you can use the programming concepts that we've covered so far: function calls, function definitions, variables and assignment
3. Practice reading the official Python documentation.
4. More practice defining your own functions and using them.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Drawing shapes

1.1 Square

1. Open the starter code `square.py`. You'll notice that a few variables are defined at the top of the file. Please use these variables in your code.
2. Add some code that moves the turtle to the (`x`,`y`) position without drawing a line.
3. Add some code that draws the square using size as the side length.
4. Make sure to save your program in a file called `square.py`.
5. Make sure that your code is fully commented.

1.2 Rectangle

1. Create a new code file called rectangle.py using a similar format to `square.py`. Meaning:
    a. Define `x, y` to indicate the starting position for the rectangle.
    b. Create: width and height to control the size of the rectangle.
    c. The entire rectangle should be visible in the turtle window without having to resize it, but beyond this restriction, feel free to place the rectangle wherever you like and to make it as big or small as you like.
2. Add code to move the turtle to position (`x`,`y`).
3. Add code to draw a rectangle with the specified width and height.
4. Save this code in a file called `rectangle.py` and make sure to comment the code.
    - Hint: It may be very helpful to copy and paste code from `square.py` to reduce the amount of typing you have to do.

1.3 Circle

1. Follow the same general steps for the rectangle, except this time you'll be creating a circle.
    a. File name: `circle.py`.
    b. Variables to create: `x, y, radius`.

1.4 Triangle

1. Follow the same general steps as for the rectangle, but this time we're making a triangle.
    a. File name: `triangle.py`.
    b. Variables to create: `x, y, size`.
2. This program should draw an equilateral triangle.
3. The size variable should indicate the length of one side.

2, Functions for shapes

1. Open the starter file `turtle_shapes.py`. This file contains the beginnings of some function definitions. (We'll be discussing function definitions more this week)
2. Copy your code for drawing a square from `square.py` into the `square(x, y, size)` function definition. There are comments explaining how to do this in the starter file.
    - Caution: Do not copy the variable definitions at the top of the square.py file. They will not be needed if you designed your square.py code correctly.
3. You should notice that the `square(x, y, size)` function is called at the bottom of the starter file. Once you've completed the previous step, running the program should produce a drawing of a square in the bottom right quadrant of the turtle window.
4. Repeat step 2 for the other shapes (rectangle, circle, triangle).
    - Caution: Don't change the order of the parameters in the function definitions.
5. Add function calls for each of the shapes at the bottom of the file to test your program and prove that the functions work as expected.
6. Please note that you should copy over comments in addition to the code itself.

3. Draw a picture

1. Create a new file and copy the function definitions from Part 2 into it. Save this new file as `my_picture.py`.
2. Write a program that uses these functions to draw a picture.
3. Picture requirements:
    a. Use at least two line colors and at least two fill colors.
    - Hint: it may be useful to refer back to your in-class assignments. If you're still having trouble, take a look at the `turtle` library section of the Python official documentation (https://docs.python.org/3/library/turtle.html)… or stop by my office hours.
    b. Your picture should depict a recognizable object or scene. For example, something like the picture bloew (though it does not need to be this involved)

c. You may add additional functions to help draw the picture (e.g. functions to draw filled-in versions of each shape), but you should not change the functions that you defined for Part 2 of the assignment.
d. Your program should use each function from Part 2 at least once, but you can also use other built-in turtle functions.

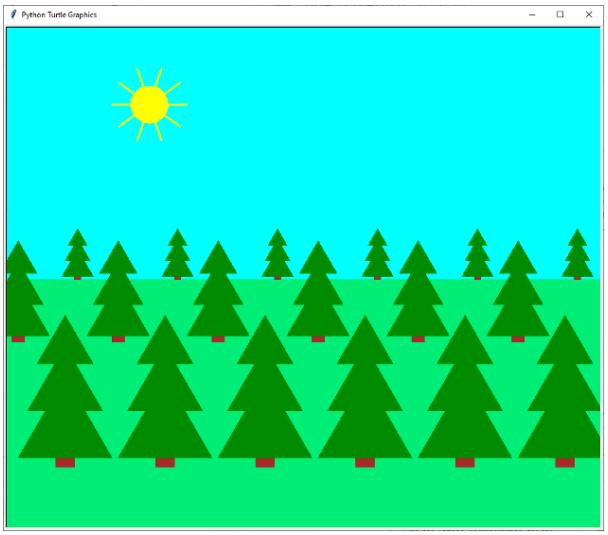


Figure 1: Turtle Drawing

4. Computer Generated art

1. Find the Python documentation for the `random` module. (Hint: Use the link to the "library reference" on the Resources page on the course website)
2. The function `randrange` from the `random` module produces a random integer. Test it out by writing a program that generates five random numbers between 2 and 9 (inclusive) and prints them out. Save your program in a file called `five_random_numbers.py`. Hint: remember to keep your code DRY.
3. Open the starter file `computer_art.py`.
4. This file already contains definitions for two functions. In particular, it contains a function called `random_shape` that randomly picks a shape (square, rectangle, circle, or triangle) and draws it. The function takes three parameters:

    a. `x` - the x location
    b. `y` - the y location
    c. `size` - the size

5. Copy and paste your function definitions from Part 2 into this file.
6. Add some code to draw 30 randomly picked shapes at random locations on the screen. For example, your output could look something like this:

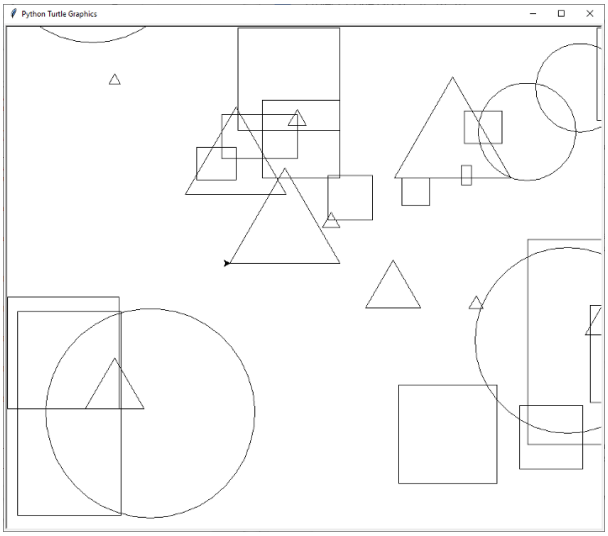


Figure 2: Computer Art

7. (Optional): Add some code to randomly change the pensize, pencolor, and filcolor.

5. What to submit

You will need to submit the following files:

1. `square.py`
2. `rectangle.py`
3. `circle.py`
4. `triangle.py`
5. `turtle_shapes.py`
6. `my_picture.py`
7. `five_random_numbers.py`
8. `computer_art.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:
    a. Header comment, with name, date, assignment and a brief description of the program.
    b. Comments within the body of the code to further clarify how the program works.
2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?
    a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running, these lines should be removed before submitting.

Once you have checked these things, submit your code to:

"Project 1: Drawings with Turtle" on Gradescope.

6. Grading guidelines

Focus on the below elements

- Correctness: programs do what the problems require.
- Program logic: use variables and functionswhere appropriate
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.
  - there are no lines of code that do not contribute to the program

Demonstration Instructions

- confirm availability for a 10 minute time slot
- submit all your code to gradescope
- bring printed versions of your code and generated images
- be prepared to answer the following questoins:

1. How did you fill in the bodies of the shape functions?
   - what python/turtle functions did you use?
   - what, if any, variables did you use (beyond the parameters specified)?
2. What additional functions did you define?
   - what are their names?
   - what, if any, parameters do they take?
   - what did you use them for?
   - how many times did you use each?
3. What variables did you define?
   - what are their names?
   - what type of values do they hold?
   - what are their purposes?
   - how many times did you use them?
   - do they get re-asssigned?
4. What will your drawing show when it is run?
   - what is the main subject?
   - what other details should I look out for?
5. What was the most challenging part of the project?
   - did you go through multiple iterations of trying to get a part to work?
   - what hint would you give to someone else attempting this project?
   - what would you do differently next time?

PF 03

CSC106: Portfolio 3

- due Friday 10/2/26 by class time
- Intro Survey
- Runestone Chapter 5
  - https://runestone.academy/runestone/default/user/register
  - your Union College email
  - unioncollege_foppff_fall26 as the course name
- Do your share-out
  - PF3 share-out schedule
  - PF3 share-out slides
- Finish all parts of the Week 3 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
- Walk through field folder
- Harvests folder
- Road repair folder

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Fri Oct. 2nd at the end of class.

Portfolio 3: Conditionals

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter(s)** (worth 0.5 of a grade percentage point):

- 5: Conditionals

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade point.*

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Remember to print and attach your programs for the coding activities. Programs should be submitted to gradescope for autograder results, but credit is giving to programs printed and included in this portfolio, on completetion, reguardless of autograder results. Programs should be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out(s):

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt: if/then	2	F 9/25		written
A2	Python (conversions)	3	F 9/25	W 9/30	GS, print
A3	Practical Practice	5	M 9/28	M 9/28	GS, print
A4	Python (painting)	7	oyo	W 9/30	GS, print
A5	Python (pay)	9	W 9/30	W 9/30	GS, print
A6	Reeborg (walk through field)	11	F 10/2	F 10/2	GS, print
A7	Reeborg (more harvests)	13	oyo	F 10/2	GS, print
A8	Written prompt: multi-branch	15	oyo		written
A9	Written study questions	16	oyo	F 10/2	written
A10	Reeborg (road repair)	18	oyo	F 10/2	GS, print
B1	Share-out report	19	oyo		written
B2	Share-out report (optional)	20	oyo		written

A1) Prompt: What is the difference between a function that prints something (or displays something, like drawing in Turtle) and a function that returns a value? Why might you want functions that return things? Frame your answer as an 'If/then' or 'if/then/otherwise' statement(s).

A2) Conversions

Save all code for this problem in a file like `conversions.py`. Don't forget to include comments for your code and to organize your code in a reasonable fashion.

1. Write a function called `ft_to_in` that converts an integer indicating a measurement in feet to inches. Your function should only do the conversion if the input is a positive number.

- (1 ft = 12 in)
- Before writing any code, answer the following questions
  - How many parameters does this function need?
  - What does each parameter stand for?
  - What datatype should the parameter(s) be?
  - Does the function have a return value?
    - If so, what is the datatype of the return value?
  - Give 2-3 example function calls and the expected output of those calls.

.....

.....

.....

.....

.....

.....

.....

.....

2. Write a function called `in_to_cm` that converts a measurement in inches to centimeters. Your function should only do the conversion if the input is less than 12 inches.

- (1 in ~= 2.54 cm)
- Think about your responses to the questions from 1. when writing this function.

3. Write a function called `cm_to_m` that converts a measurement in centimeters to meters.

- (1 m = 100 cm)
- Think about your responses to the questions from 1. when writing this function.

4. Now write a function called `main` that:

   - Asks the user to input a height in feet and inches.
   - Uses the functions you defined in 1-3 to calculate the user's height in meters.
   - At the end, the function should print out a message to let the user know what their height is in meters.

- Before writing any code, answer the following questions and include the answers as comments at the top of the file:
  - How many parameters does this function need, what do they stand for, and what datatype(s) should they be?
  - Does the function have a return value, and if so, what is its datatype?
  - Give an example function call and the expected output from the call.
- Recommendation: Have the user input the feet and inches portion of their height with two separate input calls.

.....

.....

.....

.....

.....

.....

.....

.....

.....

5. Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

6. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A3) Prompt: Based on the "Practice Practical Exam" (handed out in class), what do you think you need to study/practice for the practical? What topics do you need to reiew? What questions do you have? Where could you find the answers?

More space for study notes:

For portfolio submission, insert you printed program pages after this page.

A4) Painting

A paint store wants to build a tool to help their customers estimate how much paint they need to buy and how much labor is going to cost if they hire the store's painters to do the painting.

In your calculations, you can assume that all roms that need painting are rectangular. All walls of a room as well as its ceiling should be painted. You can disregard any windows and doors the room may or may not have. Write a program called `painting.py` that implements this tool.

1. The paint store has determined that for every 121 square feet of wall space, one gallon of paint is needed. Write a function `paint_estimator` that, given the room's dimensions (length, width, and height) in feet, returns how many gallons of paint are needed.

- What subtask do you have to solve for this problem? Define a separate function for it.
- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

.....

2. The paint store has also determined that, for every 121 square feet of wall space, 8 hours of labor will be needed. The company charges $60 per hour of labor. Define another function called `labor_estimator` that, given the room's dimensions (see previous), returns the labor cost.

- If you identified a good subtask in the first part of the problem, you should be able to re-use the function you defined for it.
- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

.....

3. Now write a function called `main` that asks the user to input the dimensions of the room (length, width, and height) in feet. The function should then calculate how many gallons of paint are needed to paint the room and how much the labor will cost. It should print out a message to provide this information to the user.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

4. (Optional) Change `main` to randomly generate the dimensions of the room instead of asking the user.

5. Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

.....

6. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A5) Overtime pay

1. Many companies pay time-and-a-half for any hours worked above 40 in a given week. Write a program named `overtime_pay.py` that includes a function called `calculate_wages` that takes two numbers indicating the hours worked (first parameter) and the hourly rate, then calculates the total wages for the week and returns them.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

2. Add some code to call the function.
3. (Optional) Add input calls to ask the user for their hours worked and hourly wage.

- Were you able to use your function above without altering it? If you did alter it, in what way? Can you imagine doing it the other way?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

4. Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

5. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

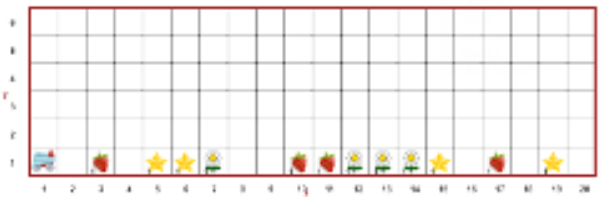
   For portfolio submission, insert you printed program pages here.

A6) Walk Through Field

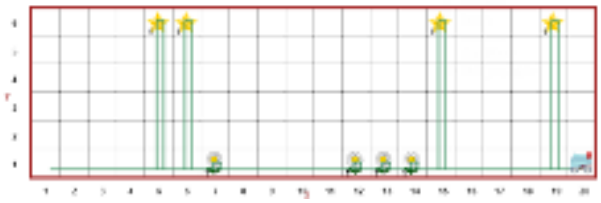
1. Download the Walk Through Field folder (or its contents) from the course website, which should contain 2 json files, which each define a single world for Reeborg.
2. Import both worlds into Reeborg and see what each world looks like.
3. Reeborg's goal is to walk along the bottom of the world and:

    a. pick all strawberries
    b. put any fallen stars back into the sky
    c. perform a dance (turn 360 degrees) for each daisy

Below are before and after pictures for Field 0.



(a) Field Before



(b) Field After

4. In english (not code), what is your algorithm for your solution?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

5. Write a program that accomplishes these goals for both worlds called `walk_through_fields.py`.

Hint: the `object_here()` function can be given a string argument to check if a specific type of object is present (e.g. `object_here("token")`).

6. Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

7. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   For portfolio submission, insert you printed program pages here.

A7) Harvests

In Python, logical operators allow you to combine two conditions into a single, more complex condition. Python has three logical operators: and, or, and not.

`<condition> and <condition>` is `True` if both conditions are `True`. `<condition> or <condition>` is `True` if at least one of the conditions is `True`. `not <condition>` is `True` the condition is `False`.

1. Download the Harvests folder from the course website and import each json file into Reeborg.
2. Find `harvest1.py` from the folder and copy the code into Reeborg.
3. Open the world Harvest 1 (from Reeborg's "introduction" worlds).
4. Run the code from `harvest1.py` and make sure you understand what it's doing.
5. Choose the world `harvest1a.json` (from the folder). You should notice that there are some holes in the field. Adapt the provided code so that Reeborg will harvest all of the carrots.

Hint: you should only have to change a small portion of the code for this and subsequent problems.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

6. In the world `harvest1b.json`. Adapt the code so that Reeborg will plant carrots in the empty spots. Name this program `harvest1b.py`.

- What will you change about your algorithm for your solution?

  .....

  .....

  .....

7. In the world `harvest1c.json`, Reeborg is carrying an infinite supply of carrots. Adapt the code so that Reeborg plants carrots in the gaps in the field. Name this program `harvest1c.py`.

- What will you change about your algorithm for your solution?

  .....

  .....

  .....

8. In the world `harvest5.json`, Reeborg's goal is to pick all of the flowers (daisies and tulips). Name this program `harvest5.py`.

- What will you change about your algorithm for your solution?

.....

.....

.....

9. In the world `harvest7.json`, Reeborg should pick everything except for the smiley tokens (Reeborg knows these as "token"). Name this program `harvest7.py`.

- What will you change about your algorithm for your solution?

.....

.....

.....

10. In the world `harvest8.json`, Reeborg should pick everything except the flowers (daisies and tulips). Name this program `harvest8.py`.

- What will you change about your algorithm for your solution?

.....

.....

.....

11. Did the versions of your programs you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithms and/or programs?

.....

.....

.....

.....

.....

12. Were you able to solve these tasks? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A8) Prompt: What is a benefit of being able to use multi-branch conditionals over simple "if" or "if..else.." statements? Give an example of how using a multi-branch statement may make your code more DRY.

A9) Check your understanding

1. What is the value of `x` after this code runs?

```
x = 5
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
```

2. What is the value of `x` after this code runs?

```
x = 1
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
```

3. Given the function definition below, what will be printed to the screen when this code runs?

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False

print(is_odd(1))
print(is_odd(2))
print(is_odd(3.5))
print(is_odd("abc"))
```

4. Consider the following function definition for is_odd. Can you rewrite the function so that it only uses one line after "def is_odd(x):"? If so, provide an example.

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False
```

5. What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
if (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

6. What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
elif (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

7. Fill in the blanks in the code below so the following payscale is followed.

		experience (years)		
		0	1-2	3+
age	<18	$15.25	$16.00	$16.00
	18+	$15.25	$16.25	$17.50

```
if experience == 0:
    wage = 15.25
elif (_1_):
    if (_2_):
        wage = 17.50
    else:
        wage = 16.25
else:
    wage = 16.0
```

8. What will the value of y be after this code runs?

```
x = 8
y = (x > 4) and ((not(x>8))or(x==5))
```

A10) Road Repair

Adjusting Reeborg's speed: Reeborg's speed can be adjusted using Reeborg's think function, which takes a single parameter `ms`, which indicates how many milliseconds Reeborg should think before taking its next action. This means that the smaller numbers will make Reeborg go faster, while larger numbers will make it go slower. The default value is 250.

1. Download the Road Repair files from the course website (3 json files for 3 Reeborg worlds) and import them into Reeborg. Each world should consist of a "roadway" with "potholes" in it. Reminder: to import a world into Reeborg using a json file, follow these steps:
    a. Click "Additional options" at the top right & Click the "Open world from file" button.
    b. Select the json file you want to import.
2. Write a program so Reeborg walks/drives along the road and fills in the potholes as it goes. Use Reeborg's `build_wall` function, which puts a wall in front of Reeborg.
3. You should write a single program that will solve all three roadX worlds in a program named `road_repair.py`.

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

6. Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

7. Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

PF 04

CSC106: Portfolio 4

- due Friday 10/9/26 by class time
- Runestone Chapter 6
- Do your share-out
  - PF4 share-out schedule
  - PF4 share-out slides
- Finish all parts of the Week 4 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
- **Submission change**
  - to get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether you progrm worked and what troubleshooting you can think of
  - you **only have to print programs you want my feedback on**
  - (you will still have to print your programs for project demos)

Starter code

- sum_of_powers.py

- Mountain Top
- number_guessing.py

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Oct. 9th by the end of class.

Portfolio 4: Indefinite Iteration

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter** (worth 0.5 of a grade percentage point):

- 6: Iteration

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point.*

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission change: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether you progrm worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on*. Printed programs can still be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/2	F 10/2	written
A2	Python (largest number, congress)	3	M 10/5	M 10/5	GS, print
A3	Written questions (a bit of review)	5	oyo	M 10/5	written
A4	Python (leap year, dice)	7	oyo	W 10/7	GS, print
A5	Written questions	9	oyo	W 10/7	written
A6	Python (cash register, sum of powers)	11	W 10/7	W 10/7	GS, print
A7	Reeborg (around the world, climbing)	13	Th 10/8	Th 10/8	GS, print
A8	Tracing while loops	15	Th 10/8	Th 10/8	written
A9	Python (primes)	17	oyo	Th 10/8	GS, print
A10	Python (number guessing)	19	oyo	Th 10/8	GS, print
B1	Share-out report	21	oyo		written
B2	Share-out report (optional)	22	oyo		written

A1) Prompt: What elements (if any) of decision making require or constitute thinking?

A2) Largest Number & Running for Congress

1. Largest number: Write a function that uses a while loop to ask the user to enter many numbers, then returns the largest number entered. The prompt asking for a number should indicate that entering -1 will cause the loop to end and the loop should, in fact, end when the user enters -1. Save all code for this problem in a file like `largest_number.py`. Don't forget to include comments for your code and to organize your code in a reasonable fashion.

    b. The main body of the program (should follow the function definition) should call the function for testing purposes.

Hint: it will probably be easier if you use more than one variable to help you keep track of the largest number entered, but you shouldn't need a lot of variables either.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

2. Running for Congress: A person is eligible to be a US senator if they are at least 30 years old and have been a US citizen for at least 9 years. To be a US representative, you need to be at least 25 years old and have been a US citizen for at least 7 years. Write a function called eligible_to_run that takes two numbers indicating a person's age and years of citizenship, then prints a message stating their eligibility for the Senate and House.

The message should have one of the following forms:

- Congratulations! You are eligible to run for the House of Representatives and the Senate.
- You are eligible to run for the House of Representatives, but not for the Senate.
- Sorry! You are not eligible to run for Congress.
- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....
- Make sure to test your implementation thoroughly. What are the different scenarios that should be tested? Show what you test by including a list of all function calls at the bottom of your file.
- (Optional 1) In addition to the citizenship and age requirements, an individual must be a resident of the state they are seeking to represent. Adjust the function you wrote to consider whether the person is a resident of the state or not.
- (Optional 2) In order to be President of the US, you need to be at least 35 years old, a natural born citizen, and have lived in the US for at least 14 years. Create a new function to check whether the person is eligible to become US president.
- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....
- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A3) A bit of review

1. The code below should print "One" if the variable `a` equals `1`, "Two" if `a` equals 2, "Three" if `a` equals 3, and "Other" if `a` is something else. But it prints "One" and "Other" if `a` equals 1, "Two" and "Other" if `a` equals 2, "Three" if `a` equals 3, and "Other" if `a` is something else. Explain what is wrong and fit it.

```
if a == 1:
    print("One")
if a == 2:
    print("Two")
if a == 3:
    print("Three")
else:
    print("Other")
```

2. Consider the following problem specification: Define a function that doubles a numeric (int or float) parameter. Which of the following successfully implement this function and demonstrate to a user that it works? For those that don't satisfy both requirements, explain what the problem(s) is/are.

```
a.  def double(a):                          ..............................
        result = 2*a                        ..............................
        return result                       ..............................
                                            ..............................
                                            ..............................
    print(result)

b.  def double(a):
        result = 2*a                        ..............................
        return result                       ..............................
                                            ..............................
    double(10)                              ..............................
    print(result)                           ..............................

c.  def double(a):
        result = 2*a
        return result                       ..............................
                                            ..............................
    double(10)                              ..............................
    print(return)                           ..............................
                                            ..............................
```

d.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(x)
```
.....
.....
.....
.....
.....

e.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(double(10))
```
.....
.....
.....
.....
.....

f.
```
def double(a):
    result = 2*a
    return result

print(double(10))
```
.....
.....
.....
.....
.....

g.
```
def double(a):
    result = 2*a
    return result

x = double(10)
print(double(10))
```
.....
.....
.....
.....
.....

h.
```
def double(a):
    result = 2*a
    print result

print(double(10))
```
.....
.....
.....
.....
.....

i.
```
def double(a):
    result = 2*a
    print result

double(10)
```
.....
.....
.....
.....
.....

A4) Leap year & Dice

1. Leap years: A year is a leap year if it is divisible by 4, unless it is a century year (divisible by 100) that is not divisible by 400. Some examples:

- 2020 was a leap year because it is divisible by 4.
- 1900 was not a leap year, because it is divisible by 100, but not 400
- 2000 was a leap year, because it is divisible by 400

Write a function that takes a year as a parameter and determines whether or not it is/was a leap year.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- (Optional) Add a while loop to the main body of the program that:
  - Asks the user to enter the year they want to check
  - Calls the function you created
  - Asks the user if they want to check another year (y/n)

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

2. Rolling Dice: Write a function that takes two parameters, faces and num_dice that indicate how many sides a single die has and how many dice it should roll respectively. (`random.randint` may be helpful here.)

b. Add two while loops that check the numbers the user enters:

- The first loop should check the number of faces and should continue running until the user provides an integer larger than one
- The second loop should check the number of dice the user will roll and should continue running until the user enters an integer larger than zero +These loops ensure that we have at least one side on each "die" and that we roll at least one die.

c. Call the function you wrote to roll the dice
d. Add another while loop that encapsulates the previous two loops as well as the call to the function.

- This loop should ask the user if they want to roll another set of dice (y/n) and continue so long as the user indicates they want to coninue
- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A5) Written Questions

1. Which of the following are expressions? Which of the expressions are Boolean expressions (ie. conditions)? What value is described by each of the Boolean expressions? Assume that the following variables have been defined:

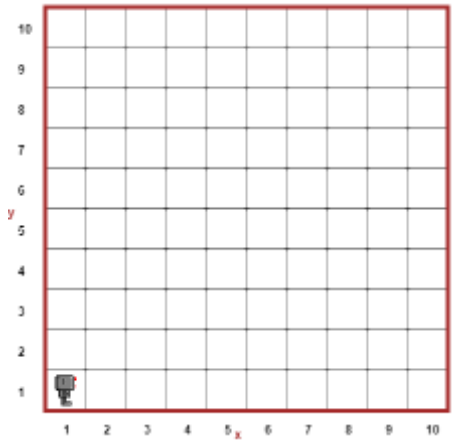
```
x = 7
y = 2
```

	expression (y/n)?	Boolean (y/n)?	value
x - 5 == y			
if x < y:			
return x			
x // 3 != 2			
x > 4			
x % 3			
x = 7			
20 = y			

2. Consider the following code snippets. For each of them, what is Reeborg going to do when this code is run? You can assume that Reeborg is carrying an unlimited supply of tokens and is starting in the bottom left corner facing East as usual.

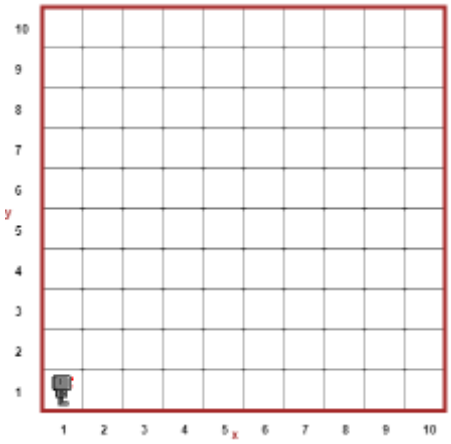
    a. Sketch the behavior of Reeborg for each of the two code snippets

```
while front_is_clear():
    put()
    move()
```



(a) a

```
while front_is_clear():
    put()
move()
```



(b) b

b. Explain why Reeborg is behaving this way in each scenario.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

A6) Cash register & sum of powers

1. Summing up numbers. Write a function called `cash_register` that repeatedly asks the user to input (positive) numbers and sums them all up. When the user inputs a negative number, the function should print out a message to let the user know their total. The negative number should not be included in the sum.

For example, one run of the function might look like this:

```
>>> cash_register()
Welcome to the cash register!
Please enter the price of each of your items.
Please enter a negative number to indicate when you're finished.

Please enter the price. (Input a negative to stop.) 2
Please enter the price. (Input a negative to stop.) 3.75
Please enter the price. (Input a negative to stop.) 9
Please enter the price. (Input a negative to stop.) 6
Please enter the price. (Input a negative to stop.) -2
Your total is: 20.75
```

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

2. Sums of Powers. Sums of powers arise in many contexts in maths. A sum of powers is a sequence of the first n integers raised to a given power. For example, the sum of the first 5 squares is: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$

   a. Download the starter file `sum_of_powers.py` from the course website.

   b. Write a function `called sum_of_powers` which, given an exponent and an integer n that indicates how many numbers to sum up, calculates (and returns) the first n integers raised to the given exponent.

   c. Save your program in a file called `sum_of_powers.py`.

d. Select a few inputs for your function and calculate what the correct output of your function should be for those inputs by hand (or with a calculator). Use these inputs to test your function.

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A7) Reeborg around the world & mountain climbing

1. Reeborg: Around the world

Use the world "Around - variable" from the "Introduction" collection of worlds. Every time the world is reloaded, it gives you a rectangular world of a different size. Additionally, Reeborg is carrying an infinite number of tokens. Write a program that makes Reeborg walk around the perimeters of the world exactly once, no matter how big the world is.

Hint: is there a way to mark the square(s) that you've been to before?

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

2. Reeborg: mountain climbing. Download the `mounatin_top` folder of starter code from the course website. It should include four different worlds, each of which starts with Reeborg at the foot of a mountain and the goal is to help Reeborg climb to the top (Note: Reeborg should climb not fly). Write a program using function definitions, loops, and conditional statements that help Reeborg climb the mountain in any of the four worlds that are given and in any other similar mountain environments.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A8) Tracing while loops

Please try to solve these problems without running/writing code. Instead you should try to come up with your answers by tracing through the code yourself and reasoning about what the Python interpreter does.

Note: You will see similar questions on the Pen and Paper Midterm.

1. What gets printed by the following code?

```
1)  x = 10
2)  while x > 8:
3)       print(x, end = " ")
4)       x = x - 1
```

.....

2. What gets printed by the following code?

```
1)  x = 10:
2)  while x > 8:
3)       x = x - 1
4)       print(x, end = " "
```

.....

3. What gets printed by the following code?

```
1)  i = 0
2)  while i < 3:
3)       print(i, end = " ")
```

.....

4. What gets printed by the following code?

```
1)  valid = False
2)  while not valid:
3)       x = int(input ("Enter a number: "))
4)       valid = (x % 2 == 1 and x % 3 == 0)
```

.....

5. After writing down your answers, check each code chunk in Python. How many were you able to sucessfully trace before running the code?

.....

6. Prompt: Is a `while` loop a way for a program to observe things? Is the program making an informed decision based on observing its current memory state (or the state of variables)?

A9) Primes

A prime number is a natural number greater than 1 that is only evenly divisible by itself and 1. In this assignment you will define a function called is_prime that checks whether a given number is a prime number.

1. Create a file called `primes.py`, which you'll use for this assignment.
2. Before writing any code, answer the following questions.

    a. How many parameters does this function need and what do they represent?
    b. Should this function have a return value? If so, what kind of value will be returned? (e.g. number, string, boolean?)
    c. Give a few example function calls that you could use to test whether your implementation works correctly.

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

3. The name of the function you create should be `is_prime`. Below is an algorithm you can use for your function.Note: You only have to worry about positive integers.

    a. Assume that x is the number that you're checking to determine if it's prime.
    b. If x is smaller than 2, it is not a prime number, so you can return False immediately.
    c. Otherwise, define a boolean variable, which represents whether or not x is prime. Initialize this variable to (i.e. assign it the value) True. Note: in effect, we begin by assuming that x is prime.
    d. For all numbers between 2 and x, check whether x is evenly divisible by the number. If so, x is not prime. Record this by changing the value of the boolean variable you defined in the previous step.

e. At the end, the value of the boolean variable should reflect whether or not x is prime, so all you need to do is return that value.

4. (Optional) Can you improve on this algorithm to make it more efficient?

Hint 1: Aside from x itself, what is the largest number x can possibly be evenly divisible by?

Hint 2: Are there any numbers between 2 and x that you don't need to check?

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A10) Number guessing

1. Your task is to write a program that plays a number guessing game. Download the starter code `number_guessing.py`. The computer chooses a random number between 1 and 100, then the user makes guesses until they find the right number. After every guess, the computer gives a hint by saying whether the number the user guessed was too high or too low.

For example, the interaction could look like this. The numbers after the colon are what the user/player types in.

So: the computer prints out `"Guess a number between 1 and 100: "`and then the user types in `50`.

```
>>> number_guessing()
Guess a number between 1 and 100: 50
That is too low. Try again: 75
That is too high. Try again: 67
That is too high. Try again: 58
That is too high. Try again: 54
That's right! My number was: 54
```

Some details:

- The computer should choose a number between 1 and 100 inclusive (i.e. [1, 100] to use math notation). That is, 1 is the smallest number the computer picks, and 100 is the largest number the computer picks. You may have to choose different start and stop points depending on which function you use from the random module.
- You should define a function called `number_guessing`. When this function is called, one whole round of the number guessing game is played. That is, when this function is called, the computer chooses a random number and then asks the user for a guess. If the guess is too high, the computer says so and lets the user input another guess; similarly if the guess is too low. This is repeated until the user guesses the right number.
- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

.....

.....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

Project 2

CSC106: Project 2

- due Monday 10/12/26 by 11:59 pm
- you may use late token(s) for this assignment
- Demo schedule
- Demo prep worksheet

In this project, you'll implement several variants of a text-based Rock-Paper-Scissors game in which a human player plays against the computer. If you're not familiar with the game, check out the Wikipedia article about it: (http://en.wikipedia.org/wiki/Rock-paper-scissors)

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or course website for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting
- This project is 5% of your final grade
- starter code:
  - rps_1.py
  - rps_2.py
  - game_history.py
  - rps_anti_human.py

- rps_human_like.py

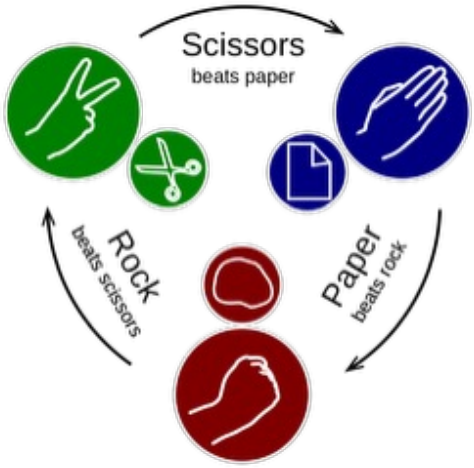
NAME:

Project 2: Rock Paper Scissors (5 pts)

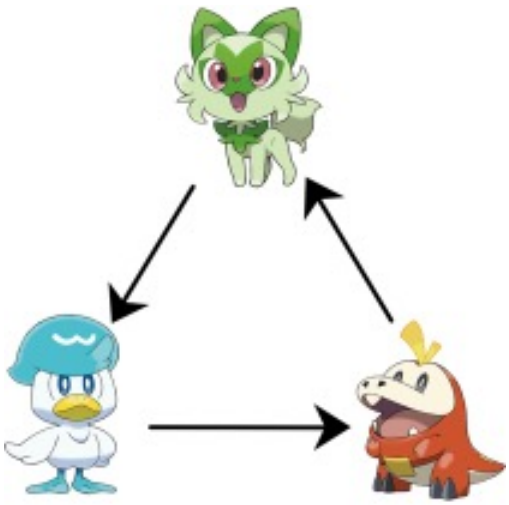
In this project, you'll implement several variants of a text-based Rock-Paper-Scissors game in which a human player plays against the computer. If you're not familiar with the game, check out the Wikipedia article about it: (http://en.wikipedia.org/wiki/Rock-paper-scissors)

Although Rock-Paper-Scissors is a simple game, mechanisms that function similarly to it have been studied in a variety of fields ranging from game theory to biology. One of the most common places to see rock-paper-scissors-like mechanics though is in video games. For example, many interactions between different "types" in the Pokémon games are similar to a game of rock-paper-scissors - the most classic of these being the interactions between the three Pokémon the player can choose at the very beginning of each game in the main series: the options being grass, fire, and water.

The version you'll implement here involves three choices - rock, paper, and scissors. Paper beats (wraps around) rock, scissors beat (cut) paper, and rock beats (blunts) scissors.



(a) RPS



(b) RPS

Learning Objectives:

1. Demonstrate that you can use the programming concepts that we've covered so far. In particular: function calls and definitions, variables and assignment, conditional statements.
2. Practice using random numbers to make random and random, but biased choices.
3. Explore ways to imitate human behavior with the computer.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Basic Rock-Paper-Scissors

1.1 Playing one game (1 pts)

1. Open the starter code `rps_1.py` from the course website.
2. Write a function called `rps` that plays a game of rock-paper-scissors with the user. The computer first randomly generates its own move (rock, paper, or scissors) and then asks the user for their choice. Next, it confirms what the user has chosen, reveals its own choice and finally makes one of the following announcements: `"It's a tie"`, `"I win"`, or `"You win"`
3. Make sure that your code is fully commented. Here are a few example interactions:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1 [Enter] <<< This is the prompt for user input.
You chose: 1 (rock)
I chose: 1 (rock)
It's a tie.
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
I chose: 2 (paper)
I win.

Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    3
You chose: 3 (scissors)
You chose: 2 (paper)
You win.
```

Hints:

- You probably will want to use the Python `random` module. You can use functions from the module that you've used before (i.e. randrange) or you could use other functions such as randint or choice.
- A link to the documentation for built-in Python modules can be found on the resources page on the course website.

- Feel free to define additional functions if you find it helpful to do so. However, please make sure that `rps` will produce one whole run through the game as shown in the example interactions above.

1.2 Playing until you're tired of it (1 pts)

1. Download the starter file `rps_2.py` from the course website. This file contains a definition for a function called `play`. When play is called with another function name as the parameter value, it will call this function repeatedly until the user indicates that they no longer want to continue.
2. Copy and paste the code for your `rps` function from `rps_1.py` into `rps_2.py`. Follow the instructions in the comments. When you run `rps_2.py`, you should see an interaction similar to the following:

```
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    2
You chose: 2 (paper)
You chose: 3 (scissors)
I win.
Do you want to play again? Type y or n.    y
Great!
Let's play Rock-Paper-Scissors!
Please type a number to make a choice:
1 -> rock
2 -> paper
3 -> scissors
What do you choose?    1
You chose: 1 (rock)
You chose: 3 (scissors)
You win.
Do you want to play again? Type y or n.    n
Ok. See you next time. Bye, bye!
```

2. RPS variations

Choose two (2) of the following four options. For each option, start by making a copy of `rps_2.py`, but save the new code under a new name, as specified by the option you're working on.

2.1 Cheating (1.5 pts)

Please use the following filename for this variant: `rps_cheating.py`.

In your original RPS program, the computer should choose its move before it asks the human player for their move. But, there's no way for the human player to check whether this is actually what's happening in the program or not. The computer could cheat by waiting for the human to choose a move, then pick a move that beats the human's move.

In this variant, that's what the computer should do. But to not be too obvious, the computer should not cheat every time. Each time the computer has to pick a move, there should be a 1/3 chance that it will cheat. Otherwise, it chooses the move randomly.

2.2 Exploiting human biases (1.5 pts)

For this variant, download the starter files `rps_anti_human.py` and `game_history.py`. Make sure to save both of these files in the same folder. You'll write all of your code in the file `rps_anti_human.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

Knowing about this behavior pattern, you can devise a strategy to beat human players:

1. If I (the computer) just lost (and my human opponent won), I know that the human is likely to play the same move they played in the last round. So I will choose my next move such that it beats their last move.
2. If I (the computer) just won (and my human opponent lost), I know that the human is likely to shift their move to beat my last move. So I will pick my move accordingly. Because of the way rock-paper-scissors works, that means I should now play whatever the human played in the last round.

To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

References: https://arstechnica.com/science/2014/05/win-at-rock-paper-scissors-by-knowing-thy-opponent/, https://arxiv.org/abs/1404.5199

2.3 Imitating human behavior (1.5 pts)

For this variant, download the starter files `rps_human_like.py` and `game_history.py`. Make sure to save both of these files in the same folder. You will write all of your code in `rps_human_like.py`.

Studies of humans playing rock-paper-scissors have shown that human players do not always pick their moves randomly. Instead, after they've won, they're more likely to repeat the same move in the next round. In contrast, when they lose, humans are more likely to pick a move to beat the move that they just lost to. For example, if they just played rock and lost to paper, they have a tendency to pick scissors in the next round in order to beat paper.

In this variant, the computer should imitate human behavior. That is:

- If the computer won the last round, it should have a tendency to repeat the same move. However, to not be too predictable, it should only do so with 50% probability. Otherwise it will pick its next move randomly.
- If the computer lost the last round, it should have a tendency to pick a move that defeats the last move by the human player. But again, to not be too predictable, it will only do so with a 50% probability. Otherwise, it will pick its next move randomly.

- If the last game ended in a tie, the computer picks the next move randomly. To implement this strategy, you need a way of remembering the outcome of the last round and what moves were made. The starter file contains a number of functions that you can use for that purpose. These functions refer to code from `game_history.py`. Please do not make any changes to `game_history.py` or to the given function definitions.

2.4 Five options: rock, paper, scissors, Spock, lizard (1.5 pts)

Start from `rps_2.py` and save it as `rps_five.py`. In this variant you will be implementing a variant of rock-paper-scissors called rock-paper-scissors-Spock-lizard. As the name suggests, it involves 5 options:

- rock beats (blunts) scissors and (crushes) lizard
- paper beats (covers) rock and (disproves) Spock
- scissors beat (cut) paper and (decapitate) lizard
- Spock beats (smashes) scissors and (vaporizes) rock
- lizard beats (poisons) Spock and (eats) paper

Make adjustments to the code originally written for `rps_2.py` so that it implements these five options instead of the original three.

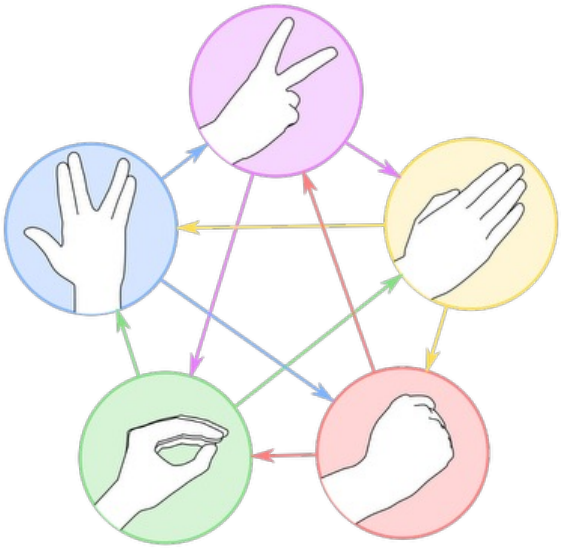


Figure 2: RPS Five

3. What to submit

What to submit You will need to submit the following files:

- `rps_1.py`
- `rps_2.py`

And two of the following (depending on which RPS variants you decide to implement):

- `rps_cheating.py`
- `rps_anti_human.py`
- `rps_human_like.py`
- `rps_five.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:

a. A header comment, which includes your name and a brief description of the program.
b. Comments within the body of the code to further clarify how the program works.

2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?

a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting. Once you have checked these things, submit your files to *Project 2: Rock-Paper-Scissors* on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

Demonstration Instructions

- TBD

PF 05

CSC106: Portfolio 5

- due Friday 10/16/26 by class time
- Runestone Chapter 7
- Do your share-out
  - PF5 share-out schedule
  - PF4 share-out slides
- Finish all parts of the Week 5 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
- **Submission**
  - to get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether you progrm worked and what troubleshooting you can think of
  - you **only have to print programs you want my feedback on**
  - (you will still have to print your programs for project demos)

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Oct. 16th by the end of class.

Portfolio 5: Definite Iteration (counting loops)

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone texbook chapter** (worth 0.5 of a grade percentage point):

- 7: More About Iteration

Note that these activities are focused on this week's theme, but they are designed to utlize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point.*

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Submission: To get credit for completing a prgramming activity, submit it to gradescope and answer the written questions about your algorithm, whether your program worked and what troubleshooting you can think of. You *only have to print programs you want my feedback on.* Printed programs can still be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/9	F 10/9	written
A2	Tracing loops	3	F 10/9	F 10/9	written
A3	Debugging with memory diagrams	4	F 10/9	F 10/9	written
A4	Prog. with Pytthon (circles, n-gons)	5	oyo	M 10/12	GS, print
A5	Math quiz	9	oyo	M 10/12	GS, print
A6	Written turtle	11	oyo	M 10/12	written
A7	Prog. with Turtle (star)	12	M 10/12	M 10/12	GS, print
A8	Written questions	13	oyo	W 10/12	written
A9	Prog. with Pytthon (stairs)	17	oyo	W 10/14	GS, print
A10	Prog. with Pytthon (retirement)	19	oyo	W 10/14	GS, print
B1	Share-out report	21	oyo		written
B2	Share-out report (optional)	22	oyo		written

A1) Prompt: What are some differences in the types of tasks that can be solved by indefinite vs definite iteration? Can you provide examples? Does one class of problems seem harder to solve than the other?

A2) Tracing loops with Memory Diagrams

Consider the following snippet of Python code.

```
1.  x = int(input("Please enter a positive integer: "))
2.  counter = x
3.  for step in range(0,10):
4.      print(counter)
5.      counter += x
```

Note: `counter += 1` does the same thing as `counter = counter + 1`

1. Draw a memory diagram after line 2 has been executed. What the user enters is up to you, but keep it in mind for the following questions.

2. Draw a memory diagram after line 5 has been executed the first time.

3. Draw a memory diagram after line 5 has been executed the second time.

4. Draw a memory diagram at the very end of the program.

A3) Debugging with Memory Diagrams

The following program is supposed to add up numbers entered by the user, but doesn't work correctly.

```
1.  steps = int(input("How many numbers do you want to add up? "))
2.  total = 0
3.  number = float(input("Please enter the next number to add: "))
4.  for step in range(0, steps):
5.      total = total + number
6.  print("Total: ", total)
```

1. Draw a memory diagram after line 2 has been executed. Assume that the user wants to add 4 numbers together.

2. Draw a memory diagram after line 5 has been executed the first time.

3. Draw a memory diagram after line 5 has been executed the second time.

4. Describe what the problem is with the program and how to fix it.

5. Copy the code into the code area of Python Tutor.
6. Click Visualize Execution and use the Next button to step through each line of code. The memory diagram will appear on the right. Check if your answers match.
7. (Optional) Repeat steps 3 and 4 for the code on pages 2 and 3.

A4) Circles & N-gons

1. Write a program to draw the following pattern:

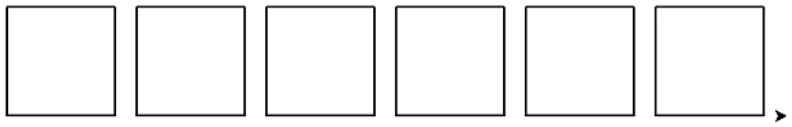


Figure 1: Turtle Squares

- Be sure to use the appropriate coding constructs.
- Note: the turtle doesn't need to end where it does in the picture.
- Can you write a function that takes the number of squares to draw as a parameter?
- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

2. Write a program to draw a circle without using the built-in `turtle.circle()` function.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

3. Circles with Turtle

a. Write turtle code to create the following:

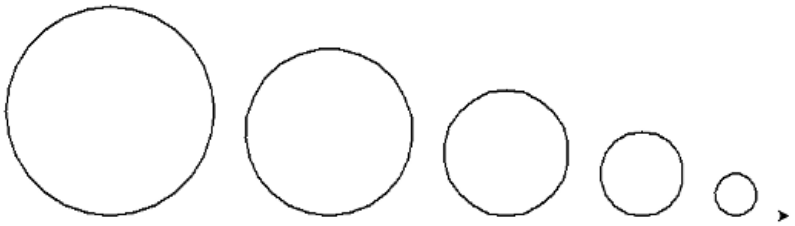


Figure 2: Turtle Circles 1

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

b. Alter that modify your code so that it makes something like this (notice the circles are now lined up through their center points):

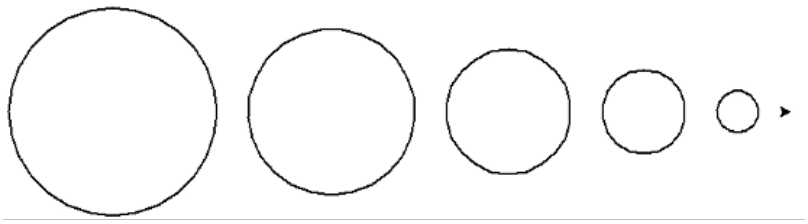


Figure 3: Turtle Circles 2

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

c. Modify the code again to make something like this:

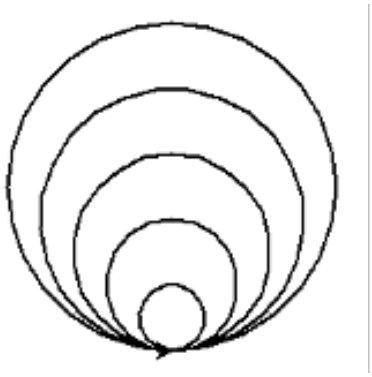


Figure 4: Turtle Circles 3

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

4. Regular n-gons with turtle: Write a program using turtle that does the following:

a. Ask the user how many sides the shape should have
b. Ask the user how long each side should be
c. Uses turtle to draw a shape with the specified number of sides, each of the specified length

- In english (not code), what is your algorithm for your function/solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A5) Math facts quiz

4. Write a function `math_quiz` that helps elementary students practice their math facts.

- Ask the user to input two numbers
- Ask the user which number is greater
- If the user's answer is right, print :

```
"Correct!"
```

- If the user's answer is wrong, print:

```
"Incorrect, the correct answer is <the correct value>!"
```

5. In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

6. (Optional) Add the functionality to randomly generate the two numbers instead of user input. Present the numbers to the user, and then wait for the user to input a result.

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

7. (Optional) If the user's result is correct, the function should print out a message indicating as much. Otherwise, it should say that the answer was incorrect and show the right answer.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

Give your programs meaningful names (ending in `.py`) and submit to Gradescope.

8. Did the autograder(s) produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

9. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

For feedback, insert you printed program(s) here.

A6) Written Turtle

1. Consider the following Python program:

   a.
```
import turtle

for size in range(10, 60):
    for side_count in range(0, 4):
        turtle.forward(size)
        turtle.left(90)
```

   b. Draw what shape the turtle draws when the inner loop is run once. Your sketch does not have to be to scale (any size is ok).

   c. Sketch what shape the turtle draws when the whole program is executed.

   d. Explain your answer to (b). How do the outer loop and the inner loop work together to produce the picture you sketched?

   .....

   .....

   .....

   .....

   .....

2. How can we translate this code into a while loop?

```
1)  for i in range(n):
2)        <body>
```

.....

.....

.....

A7) Turtle Star

1. Write a program using the turtle module to draw the following picture.

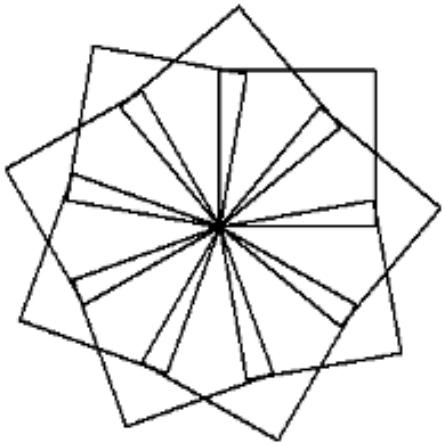


Figure 5: Turtle Star

- Before writing any code, in english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

A8) Written Questions

1. Consider the following (incomplete) Python program:

```
x = int(input("Please input a positive integer: "))
counter = x
for step in range(0, 10):
```

Complete the missing lines of code above so that the program will produce the following output if the user inputs 3.

```
3
6
9
12
15
18
21
24
27
30
```

- **Notes**: You do not need to introduce any additional variables. There is more than one way to achieve this result.

2. Consider the following (incomplete) Python program to list the first few Fibonacci numbers:

The Fibonacci numbers are the following sequence of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …

- The first two numbers are 0 and 1.
- After that, each number can be calculated by adding the previous two together. For example:
  - the third number is 1 (0+1)
  - the fourth number is 2 (1+1)
  - the fifth number is 3 (2+1), etc.

They are named Fibonacci numbers after the Italian mathematician who first described them in Europe. Patterns based on the Fibonacci sequence show up in surprisingly many contexts in math, biology, and art. (See the Wikipedia entry if you want to know more.)

Below is the (incomplete) Python program to list the first few Fibonacci numbers.

The user gets to determine how many Fibonacci numbers to list.

```
1.  print ("How many Fibonacci numbers would you like to see?")
2.  limit = int (input("Please input a number greater than 2: "))
3.
4.  fib_0 = 0
5.  fib_1 = 1
6.
7.  print(fib_0)
8.  print(fib_1)
9.  for count in range(        ):
10.    next_fib =
11.    print(next_fib)
```

- For questions a-d, assume that the program has been completed and functions correctly.

2.a. Draw a memory diagram that shows the- associations after line 5 has been executed. Assume that the user inputted 10. Also show what will have been printed to the screen at that point.

2.b. Draw a memory diagram that shows the - associations that should exist after line 11 has been executed for the first time. Also show what line 11 will print to the screen when it is executed for the first time.

2.c. What should the - associations be when line 11 has been executed for the second time? Draw a memory diagram.

2.d. Draw a memory diagram that shows the - associations after the last number has been printed (i.e. at the end of the program).

2.e. Complete the code above so that the program prints the first n Fibonacci numbers, where n is determined by the user. You may need to add new lines of code.

3. Consider two Python programs:

- Both programs on the next page will help Reeborg harvest the carrots in the Harvest 1 world:

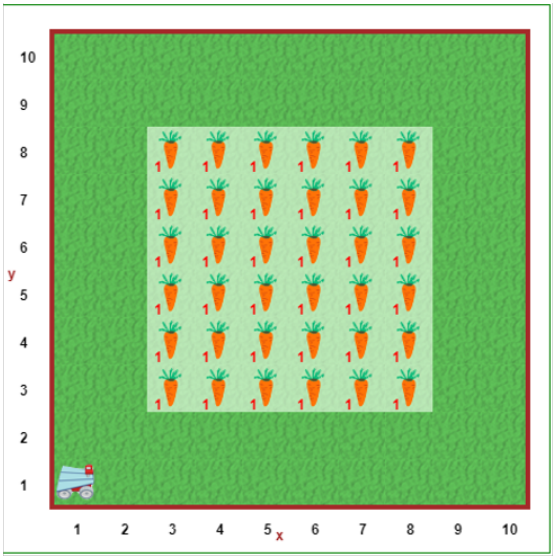


Figure 6: Reeborg Carrots

- Both programs define a number of functions. Unfortunately, the program designers were from the bad old days when names needed to be as short as possible, so their function names are essentially meaningless. Come up with better names for each of the functions that they defined.

a. .....
b. .....
c. .....
d. .....
e. .....
f. .....
g. .....
h. .....

3.i **After** you've come up with better names for the functions, which of the solutions do you think is better and why?

.....

.....

.....

.....

.....

.....

.....

```
# Program 1

def turn_right():
 turn_left()
 turn_left()
 turn_left()

def a():
 move()
 turn_left()
 move()
 move()
 turn_right()
 move()

def b():
 for carrot in range(0, 6):
     take()
     move()

def c():
 turn_left()
 move()
 turn_left()
 move()

def d():
 turn_right()
 move()
 turn_right()
 move()

a()
for double_row in range(0, 3):
 b()
 c()
 b()
 d()
```

```
# Program 2

def turn_right():
 turn_left()
 turn_left()
 turn_left()

def e():
 move()
 turn_left()
 move()
 move()
 turn_right()
 move()
 take()

def f():
 for carrot in range(0, 5):
     move()
     take()

def g():
 f()
 turn_left()
 move()
 turn_left()
 take()

def h():
 f()
 turn_right()
 move()
 turn_right()
 take()

e()
for double_row in range(0, 2):
 g()
 h()
g()
f()
```

A9) Turtle Stairs

1. Create a file called square_chain.py for your code.
2. Write a Python program using turtle to draw something that draws a diagonal chain of 10 squares:

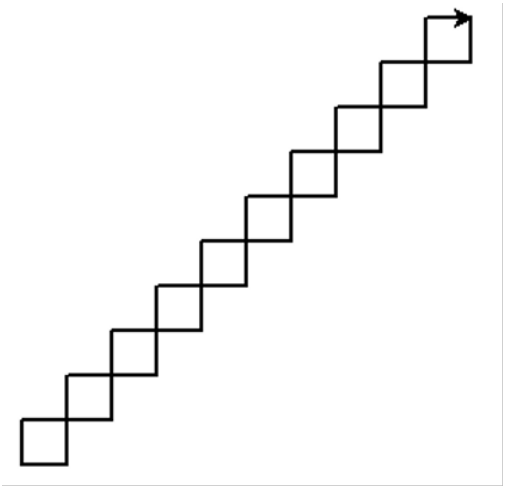


Figure 7: Turtle Stairs

There are many different ways (algorithms) that you can draw this pattern. Some may (or may not) leave the turtle in the same position and orientation that you see here. It does not matter if the turtle has the same position and orientation as what you see here.

3. Add comments to explain how your program works.
    - Header comment with Name, Email, Date, Assignment, Description
    - In-line comments describing the algorithm guiding your program
    - Docstrings for any functions, if you define new functions

- In english (not code), what is your algorithm for your solution?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

.....

.....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  .....

For feedback, insert you printed program(s) here.

A10) Retirement Calculator

It's never too early to start saving for retirement. For this assignment you're going to write a simple retirement calculator.

- A run of the program may look like the following (user input in bold):

```
How much do you plan to save each month? (in dollars) **100**
How old are you? (in years) **20**
At what age do you plan to retire? (in years) **40**
What yearly rate of return do you expect? (in percentage points) **5**
Your savings at age 21 are: $ 1200
Your savings at age 22 are: $ 2460
Your savings at age 23 are: $ 3783
Your savings at age 24 are: $ 5172
Your savings at age 25 are: $ 6630
Your savings at age 26 are: $ 8162
Your savings at age 27 are: $ 9770
Your savings at age 28 are: $ 11458
Your savings at age 29 are: $ 13231
Your savings at age 30 are: $ 15093
Your savings at age 31 are: $ 17048
Your savings at age 32 are: $ 19100
Your savings at age 33 are: $ 21255
Your savings at age 34 are: $ 23518
Your savings at age 35 are: $ 25894
Your savings at age 36 are: $ 28388
Your savings at age 37 are: $ 31008
Your savings at age 38 are: $ 33758
Your savings at age 39 are: $ 36646
Your savings at age 40 are: $ 39679
```

1. Start by creating a file called retirement.py for your code.
2. Next, start writing code to get input from the user and store the information in variables.

- The program should start by asking the user for the following information:
  - The amount they plan to save each month
  - Their starting age
  - Their planned retirement age
  - The yearly rate of return they expect (i.e. growth through interest or dividends).

3. Now design an algorithm to compute the amount that accumulates each year. Write out the algorithm in a human language (e.g. English) before writing it in code. Include this as your description in your header comment.

- The program should show how the user's savings grow over the course of their careers. For example, if a 20 year-old starts saving $100 a month, retires at 40, and gets a 5% rate or return, then they should have about $39,679 by the time they retire.

- To simplify things, we assume that interest is added once per year before the user has contributed any of their monthly savings.
  - This means that, for the first year, the rate of return is zero, so they only get what they saved over the course of the year (i.e. $1200).
  - For subsequent years, calculate the interest gained, then add the yearly contribution. So for year 2:
    * the interest will be $1200 × 5% = $60
    * the savings at the end of year will be $1200 + $60 + $1200= $2460.

4. Implement the program in Python
5. Test the program with different values to ensure that it works correctly.
6. If you haven't already, add comments explaining how the program works.

- In english (not code), what is your algorithm for your solution?

  .....

  .....

  .....

  .....

  .....

  .....

  .....

  - Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

  .....

  .....

  .....

  .....

  .....

- Were you able to solve this task? If not, can you think of anything else to try?

  .....

  .....

  .....

  .....

  .....

  .....

  For feedback, insert you printed program(s) here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

Weekly Notes

Week 1

Notes from Week 1

- Wed
  - Seymour Pappert Turtle Documentary
  - Activities for Portfolio 1: A1 (prompt) and A2 (turtle drawing)
- Thurs
  - sort ourselves on "Can computers think"?
  - log onto computers: log on instructions
  - register for Runestone: see Resources page
  - also maybe helpful, a different Runestone book
- Fri
  - Reeborg programs (portfolio A4 and A5)
  - computer basics answers (portfolio A3): share-out
- Mon
  - Reeborg share-outs (sched, slides)
  - discussion on functions, parameters, comments
  - variables and assignments
  - first Python program: hello world
  - python practice with arithmetic (PF A8)
- computers "thinking" in the news

  - The Navier-Stokes problem solved
  - American Mathematical Society statement on NS solution
  - "Misleading Metaphors and Real Risks"

look ahead to Wed

- Wednesday
  - more share-outs on syntax vs semantics (PF A6) and variable assignment tracing (PF A7)
  - python practice with Turtle module (PF A10)
  - look ahead to Project 1
  - begin Week 2, PF2

finish up rest of PF by Thursday

- any remaining share-outs for PF1

Week 2

Wed (9/16)

- Finish up share-outs, (slides)
- Updated Winding Sandbar autograder (allows for `done()`)
  - ask if you are curious why!
- This week, extend portfolio deadline until Thursday (9/17) at the end of class

Thurs (9/17)

- Turtle activity from PF1 (A10)
- Find a way to share-out if you haven't (Turtle, arithmetic)
- Project 1
- PF 2

Fri (9/18)

- Jump to A3, Turtle Flowers First
- then do A2, Reeborg (time permitting)
- stars0.json
- stars1.json

Mon (9/21)

- PF2, starting with prompt
- PF2 share-outs: new slides, new schedule
- A couple notes about PF1
- Functions - input/output
- PF2 A4 **but SKIP questions 4-9**

Key Resources this week

- Updated python download link (https://www.python.org/downloads/)
- all also found in Resources page
- Python Documentation
- Python Tutor

Examples

- ascii_art.py

Week 3

PF3 share-outs

- PF3 share-out schedule
- PF3 share-out slides

Key Resources this week

- Python Documentation
  - including `turtle` documentation
- Python Tutor
- Runestone through chapter 5!

Starter code

- Walk through field folder
- Harvests folder
- Road repair folder

Wed (9/23)

- CS Helpdesk intro
- A couple notes about PF1
- Functions - input/output

- PF2 A4 **but SKIP questions 4-9**
- PF2 share-outs: new slides, new schedule

Thur (9/24)

- PF2 A5 & A6
- Week 2 share outs

Fri (9/25)

- Finish Week 2 share-outs
- collect PF2
- Begin conditionals! (PF3 - conversions)

Mon (9/28)

- Practical Exam Practice
- Project 1 due
  - demo time slots
  - move yourself to an empty slot if you need to
  - demos in Steinmetz 209A
- continue with conditionals / PF3

Examples

- char_shape.py

Week 4

PF3 share-outs

- PF3 share-out schedule
- PF3 share-out slides

Starter code

- Walk through field folder
- Harvests folder
- Road repair folder

Wed (9/30)

- continuing with conditionals
- overtime pay activity (PF3 A5)
- share-outs (PFA2, AA4, A5)
- ascii-shape hints

Thur (10/1)

- Practical Exam 1
  - 1.5 hours, this classroom (or testing center)
  - topics through PF1 & PF2, Runestone through Ch. 4

  - topics from PF3 (conditionals) allowed but not neccesary
  - coding constructs beyond these topics **not** allowed

Fri (10/2)

- Finish Week 3 share-outs
- collect PF3
- hand out PF4: Indefinite iteration (loops)!
  - begin by reading runestone chapter 6
  - PF4 share-out schedule
  - PF4 share-out slides

Mon (10/5)

- Begin PF4 (Conditional Loops)!
- PF4 share-outs
- PF4 A2: largest number & running for congress

Week 5

Wed 10/7

- Continuing with while loops!
- PF4 A6: cash register & sum of powers
- PF4 share-outs
  - PF4 share-out schedule
  - PF4 share-out slides

Thurs 10/8

- finish PF4 share-outs
- time to work on Project 2: Rock, Paper, Scissors
- demo schedule TBD

Fri 10/9

- PF5: Definite Iteration (for loops)
  - will be on the midterm next week (Thurs 10/15)

Mon 10/12

- PF5 cont.
- Proj 2 demos

Week 6

Wed 10/14

- PF5 share-outs
  - PF5 share-out schedule
  - PF5 share-out slides
- review for exam
  - Extra Review PDF
  - Extra Review Python notebook

Thurs 10/15

- Midterm Exam (written)
- 1.5 hours

Fri 10/16

- finish PF5 share-outs, if needed
- PF6
  - share-out sched: TBD

Mon 10/19

- PF6: Developers as Decision Makers Workshop

About

Me:

- Georgia Doing, PhD
- she/her/hers
- My interests: computational microbiology, small cells & big data (check out my github website for descriptions of research projects)
- My dog Ginny (just email ahead of office hours if you'd like me to make sure she isn't there, no problem!)

References